

12

CS336 Assignment 5 (alignment): Alignment and Reasoning RL

12.1. 拒绝采样

参考资料 12.1

<https://blog.csdn.net/jteng/article/details/54344766>

拒绝采样 (Rejection Sampling) 是蒙特卡洛采样法中的一个重要方法, 它的核心目的是在无法直接对目标分布进行采样时, 通过对一个易于采样的辅助分布进行采样, 并依据特定概率“拒绝”掉一部分样本, 从而间接获得服从目标分布的样本。

12.1.1. 核心定义与符号体系

- **x**: 随机变量, 可以是标量也可以是高维向量, $x \in \mathbb{R}^d$ 。
- **p(x)**: **目标分布** 的概率密度函数 (PDF).
 - 在实际工程中, $p(x)$ 通常难以直接归一化, 我们往往只能计算出近似归一化的 $\tilde{p}(x)$, 即 $p(x) = \frac{1}{Z} \tilde{p}(x)$, 其中 Z 是未知的归一化常数。
- **q(x)**: **提议分布**。这是一个易于采样的分布 (如高斯分布、均匀分布)。
- **k**: **比例常数**。它是一个标量, 且必须满足 **包络条件**: 对于定义域内所有的 x , 都有 $k \cdot q(x) \geq \tilde{p}(x)$ 。
- **u**: **辅助随机变量**, 服从均匀分布 $u \sim \text{Uniform}[0, 1]$ 。

12.1.2. 直觉上理解

通过下面的概率分布图可以很好理解拒绝采样的想法。

- **红色曲线**: 表示归一化目标分布的概率密度函数 $\tilde{p}(z)$, 它比较复杂, 难以直接采样得到。
- **蓝色曲线**: 表示乘上比例系数的提议分布函数 $k \cdot q(x)$, 它可以被我们表示、采样出来, 图中以高斯分布为例。

首先可以看到, 蓝色曲线是完全包络红色曲线的, 即 $k \cdot q(x) \geq \tilde{p}(x)$, 这表示 **目标分布的概率密度函数是完全可以在我们的提议分布这一小范围内得到**。

接下来, 我们采样的时候, 比如选择 z_0 , 那么对应的 $k \cdot q(x)$ 结果就是 $k \cdot q(z_0)$; 我们知道在这一条纵轴上 $\tilde{p}(z_0)$ 的结果一定是在 $[0, k \cdot q(z_0)]$ 之内的, 因为目标分布被包络了。所以拒绝采样的核心想法就是在 $[0, k \cdot q(z_0)]$ 之上 **均匀分布**, 得到的

12.1	拒绝采样	165
12.1.1	核心定义与符号体系	165
12.1.2	直觉上理解	165
12.1.3	严谨数学证明	166
12.2	重要性采样	168
12.2.1	核心定义与符号体系	168
12.2.2	数学原理推导	168
12.2.3	举例理解	169
12.2.4	PPO 的重要性采样应用	170
12.3	PPO 前身: Vanilla Policy Gradient 和 TRPO 数学原理超详细推导	171
12.3.1	优势函数 A^π 知识补充	171
12.3.2	Vanilla Policy Gradient (朴素策略梯度)	171
12.3.3	TRPO	174
12.4	GAE	181
12.4.1	省流版	181
12.4.2	核心定义与符号体系	181
12.4.3	问题背景	181
12.4.4	数学原理推导	181
12.4.5	直觉理解	182
12.5	PPO	186
12.5.1	核心定义与符号体系	186
12.5.2	数学原理推导	186
12.6	GRPO	189
12.6.1	核心定义与符号体系	189
12.6.2	数学原理推导	189
12.6.3	直觉理解与物理意义	191
12.7	第三部分 Measuring Zero-Shot MATH Performance	192
12.7.1	实现细节	192
12.8	第四部分 SFT Helper	193
12.8.1	作业 1 'tokenize_prompt_and_output' 要求:	193
12.8.2	作业 2 'com-	

结果 u_0 如果小于 $\tilde{p}(z_0)$, 证明在红色曲线目标分布面积之内, 采样有效, 否则表示在红蓝之间的灰色面积里, 采样无效, 拒绝。

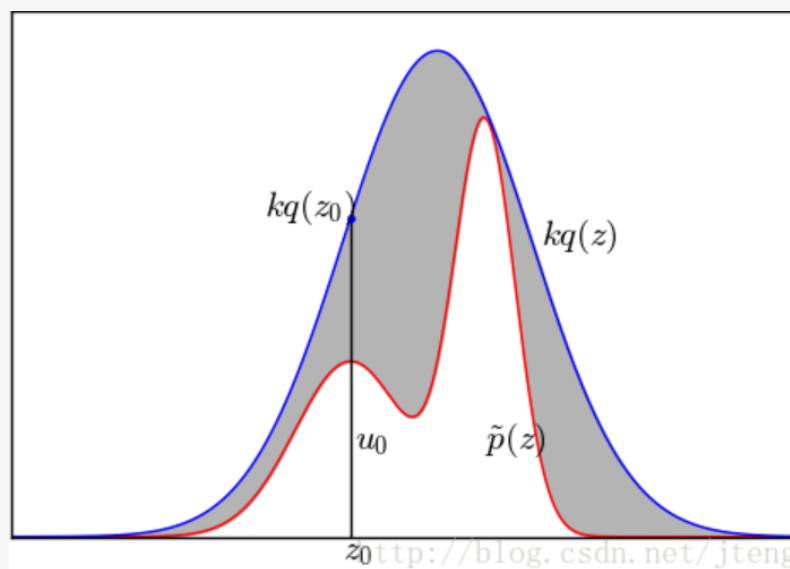


图 12.1: 拒绝采样示意图

12.1.3. 严谨数学证明

我们的目标是证明:通过拒绝采样算法生成的样本, 其真实的概率分布确实是 $p(x)$ 。

算法流程

1. 从**提议分布**中采样一个候选样本: $X \sim q(x)$ 。
2. 从**均匀分布**中采样一个辅助变量: $U \sim \text{Uniform}[0, 1]$ 。
3. 接受判据: 如果 $U \leq \frac{\tilde{p}(X)}{k \cdot q(X)}$, 则接受 X ; 否则拒绝 (丢弃并重试)。

证明过程

我们需要计算在“接受” (记为事件 A) 的条件下, 样本 X 的**边缘概率密度**。根据贝叶斯定理:

$$p(x|A) = \frac{p(A|x) \cdot p_{\text{candidate}}(x)}{p(A)}$$

这里有三项关键需要展开:

1. **步骤 1: 分析 $p_{\text{candidate}}(x)$** 由于候选样本 X 是直接来自 $q(x)$ 中采样的, 所以其先验概率密度就是: $p_{\text{candidate}}(x) = q(x)$
2. **步骤 2: 分析条件接受概率 $p(A|x)$** 给定一个具体的 x , 我们接受它的条件是 $U \leq \frac{\tilde{p}(x)}{k \cdot q(x)}$ 。因为 U 是 $[0, 1]$ 上的均匀分布, 所以该事件发生的概率就是区间的长度:

$$p(A|x) = \mathbb{P}\left(U \leq \frac{\tilde{p}(x)}{k \cdot q(x)}\right) = \frac{\tilde{p}(x)}{k \cdot q(x)}$$

3. **步骤 3: 计算总接受率 $p(A)$** 这是所有可能得 x 被接受的概率积分:

$$\begin{aligned} p(A) &= \int p(A|x) \cdot q(x) dx \\ &= \int \left(\frac{\tilde{p}(x)}{k \cdot q(x)} \right) \cdot q(x) dx \\ &= \int \frac{\tilde{p}(x)}{k} dx \\ &= \frac{1}{k} \int \tilde{p}(x) dx \end{aligned}$$

由于 $p(x) = \frac{\tilde{p}(x)}{Z}$, 故 $\tilde{p}(x) = Z \cdot p(x)$ 。

$$p(A) = \frac{1}{k} \int Z \cdot p(x) dx = \frac{Z}{k} \underbrace{\int p(x) dx}_1 = \frac{Z}{k}$$

4. **步骤 4: 整合最终结果** 将上述三项代入贝叶斯公式:

$$\begin{aligned} p(x|A) &= \frac{\left(\frac{\tilde{p}(x)}{k \cdot q(x)} \right) \cdot q(x)}{\frac{Z}{k}} \\ &= \frac{\frac{\tilde{p}(x)}{k}}{\frac{Z}{k}} \\ &= \frac{\tilde{p}(x)}{Z} \\ &= p(x) \end{aligned}$$

12.2. 重要性采样

参考资料 12.2

<https://zhuanlan.zhihu.com/p/41217212>

<https://zhuanlan.zhihu.com/p/342936969>

重要性采样 (Importance Sampling) 也是**蒙特卡洛方法**的一种**在线策略 (On-policy)**与**离线策略 (Off-policy)**上有重要作用。

重要性采样解决的核心问题是：当我们无法直接从目标分布中采样，或者为了降低方差而故意从另一个分布中采样时，如何无偏地估计原分布下的期望值。

12.2.1. 核心定义与符号体系

- $x \in \mathcal{X}$: 随机变量 (例如：强化学习中的轨迹、图像生成中的噪声向量)。
- $p(x)$: **目标分布**。这是我们真正关心的分布，我们希望计算关于它的统计量。
- $f(x)$: 我们关心的**目标函数**(例如：奖励函数、损失函数)。我们想计算的是期望 $\mathbb{E}_{x \sim p}[f(x)]$ 。
- $q(x)$: **提议分布**。这是我们实际用来进行采样的分布 (通常 $q(x)$ 比 $p(x)$ 更容易采样，或者能采到更多“重要”样本)。
- $w(x)$: **重要性权重**，一般定义为 $w(x) = \frac{p(x)}{q(x)}$ 。

12.2.2. 数学原理推导

我们的目标是计算函数 $f(x)$ 在目标分布 $p(x)$ 下的期望值: $I = \mathbb{E}_{x \sim p}[f(x)]$

积分形式展开

根据期望的定义，将其写为积分形式: $I = \int_{\mathcal{X}} f(x)p(x) dx$

引入提议分布

假设 $q(x)$ 是一个已知且易于采样的分布。我们在积分内部同时乘以并除以 $q(x)$ 。**数学公理约束**: 为了保证这一步合法，必须满足**绝对连续性**，即对于任意 x ，若 $p(x)f(x) \neq 0$ ，则必须有 $q(x) > 0$ 。简言之，**q 的支撑集 (Support) 必须覆盖 p 的支撑集**。

只要 $p(x)$ 认为某件事有可能发生 (概率 > 0)，那么 $q(x)$ 也必须认为这件事有可能发生 (概率 > 0)。

$$I = \int_{\mathcal{X}} f(x) \frac{p(x)}{q(x)} q(x) dx$$

重组为新的期望

我们将 $\frac{p(x)}{q(x)}$ 视为样本 x 的权重，记为 $w(x)$ 。此时，积分项 $q(x)dx$ 代表了我们在 q 分布下的概率测度。于是，原积分转化为在分布 $q(x)$ 下的新期望：

$$I = \mathbb{E}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] = \mathbb{E}_{x \sim q} [f(x)w(x)]$$

蒙特卡洛估计

在实际计算中, 我们无法计算解析解积分, 只能通过从 $q(x)$ 中采样 N 个样本 $\{x_i\}_{i=1}^N$ 来进行离散近似:

$$\hat{IIS} = \frac{1}{N} \sum_{i=1}^N f(x_i) \frac{p(x_i)}{q(x_i)}$$

这就是**重要性采样估计量**。它证明了: **我们可以通过采样自己提出的提议分布 q , 并通过权重 w 修正, 从而得到 p 分布下的无偏估计。**

缺陷: 方差分析

方差是重要性采样分析的重要缺陷。估计量的方差为: $\text{Var}_{x \sim q}[\hat{IIS}] = \frac{1}{N} \text{Var}_{x \sim q} \left(f(x) \frac{p(x)}{q(x)} \right)$ 。如果 $q(x)$ 在某些 $p(x)f(x)$ 很大的区域取值很小 (即 $q(x)$ 是轻尾的), 那么权重 $w(x) = p(x)/q(x)$ 会爆炸式增长, 导致估计量的方差趋于无穷大。这就是为什么选择合适的 $q(x)$ 至关重要。

12.2.3. 举例理解

任务目标

假设 x 服从标准正态分布 $x \sim \mathcal{N}(0, 1)$ 。我们想计算 x 落在 3 之外的概率: $P(x > 3) = \mathbb{E}_{x \sim p}[\mathbb{I}(x > 3)]$

这里:

- **目标分布** $p(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$
- **目标函数** $f(x) = \mathbb{I}(x > 3)$ 。这是一个**指示函数**: 当 $x > 3$ 时为 1, 否则为 0。
- **真实值**: 根据统计学表, 这个概率约为 **0.00135**(即千分之 1.35)。

朴素蒙特卡洛采样

如果我们直接老老实实地从 $p(x)$ (标准正态分布) 中采样:

因为绝大多数 x 都会落在 $[-2, 2]$ 之间。所以采了 100 次, 很可能所有样本都小于 3, 结果 $\hat{I} = 0$ 。

而且**方差极大**: 你要么算出来是 0, 要么因为偶尔遇到一个样本而剧烈波动。

重要性采样

为了解决“采不到”的问题, 我们故意从一个更容易产生 $x > 3$ 的分布中采样, 比如均值为 4 的正态分布。

1. 设计提议分布 $q(x)$

设 $q(x) \sim \mathcal{N}(4, 1)$ 。

- 在这个分布里, x 很容易大于 3(因为中心就在 4)。
- 它的支撑集是 $\mathbb{R}$, 覆盖了 $p(x)$ 的支撑集, 满足绝对连续性。

2. 推导权重 $w(x)$

对于任意采样到的样本 x ，我们需要计算权重 $w(x) = \frac{p(x)}{q(x)}$ 。

$$p(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$

$$q(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(x-4)^2}{2}\right)$$

代入计算权重 (常数项约掉): $w(x) = \frac{\exp(-x^2/2)}{\exp(-(x-4)^2/2)} = \exp\left(-\frac{x^2}{2} + \frac{(x-4)^2}{2}\right)$

展开指数部分: $-\frac{x^2}{2} + \frac{x^2-8x+16}{2} = \frac{-8x+16}{2} = -4x+8$ 所以, 权重的解析式极为简洁: $w(x) = e^{8-4x}$

3. 实际计算流程

假设我们进行一次采样:

(a). **采样**: 从 $q(x) \sim \mathcal{N}(4, 1)$ 中抽取一个样本。假设抽到了 $x = 3.5$ 。

(b). **计算目标函数**:

$$f(3.5) = \mathbb{I}(3.5 > 3) = 1$$

(c). **计算权重**: $w(3.5) = e^{8-4(3.5)} = e^{8-14} = e^{-6} \approx 0.00248$

(d). **最终贡献**: 这个样本对期望的贡献是 $1 \times 0.00248 = 0.00248$ 。

结果分析: 请注意, 这个单次样本的估计值 0.00248 已经非常接近真实值 0.00135 了。相比之下, 朴素方法采样一次得到的通常是 0。IS 极大地提高了采样效率。

12.2.4. PPO 的重要性采样应用

在 PPO 的实际训练中, 我们手里只有旧策略 $\pi_{\theta_{old}}$ 采样的数据。为了计算新策略 π_{θ} 的期望, 我们必须使用重要性采样 (IS) 变换。

$$L^{CPI}(\theta) = \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t \right]$$

这就是 TRPO/PPO 中那个比率 $\frac{\pi_{\theta}}{\pi_{\theta_{old}}}$ 的由来。它本质上是在修正用旧策略数据评估新策略表现时的概率偏差。

12.3. PPO 前身:Vanilla Policy Gradient 和 TRPO 数学原理超详细推导

参考资料 12.3

<https://www.cnblogs.com/xingzheai/p/16565686.html>

无论是朴素梯度策略,TRPO 还是比较流行的 PPO、GRPO、DPO, 它们都有一个共同点, 那就是它们的关注点是**策略函数** $\pi_\theta(a|s)$, 而不是像 DQN 这种算法先去学一个价值函数 $Q(s, a)$ 再反推策略。

12.3.1. 优势函数 A^π 知识补充

- $V^\pi(s)$: 状态价值函数, 表示从状态 s 开始, 遵循策略 π 所能获得的期望累积回报
- $Q^\pi(s, a)$: 动作价值函数, 表示在状态 s 采取动作 a , 之后遵循策略 π 所能获得的期望累积回报
- $A^\pi(s, a)$: 优势函数, 定义为动作价值函数与状态价值函数之差: $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ 。(不过在 TRPO、PPO 这种关注策略的方法里我们只能看到轨迹回报, 无法得到 $Q(s, a)$ 和 $V(s)$ 这些**期望值**, 因此采用的方式是**GAE**来得到 A)

将 $Q^\pi(s, a)$ 的贝尔曼方程代入优势函数定义:

$$\begin{aligned} A^\pi(s, a) &= Q^\pi(s, a) - V^\pi(s) \\ &= \mathbb{E}_{s'}[r(s, a, s') + \gamma V^\pi(s')] - V^\pi(s) \\ &= \mathbb{E}_{s'}[r(s, a, s') + \gamma V^\pi(s') - V^\pi(s)] \end{aligned}$$

这个形式揭示了优势函数的本质:**它衡量的是采取动作 a 后获得的即时奖励加上未来价值, 相对于当前状态平均价值的增量。**

一个重要的数学性质是, 优势函数在策略 π 下的期望为零:

$$\begin{aligned} \mathbb{E}_{a \sim \pi(\cdot|s)}[A^\pi(s, a)] &= \sum_a \pi(a|s) A^\pi(s, a) = \sum_a \pi(a|s) [Q^\pi(s, a) - V^\pi(s)] \\ &= \sum_a \pi(a|s) Q^\pi(s, a) - V^\pi(s) \sum_a \pi(a|s) = V^\pi(s) - V^\pi(s) = 0 \end{aligned}$$

这个性质表明:**优势函数是一个“零中心化”的量, 它只关注相对好坏, 而非绝对价值。**

直观理解

可以在以股市买股票打比方 ($\times$)

- $V^\pi(s)$: 是你随便买股票 (按照当前策略随机选择方向) 最终赚到的钱。
- $Q^\pi(s, a)$: 是你按照自身的判断只买特定的股票 (比如买茅台) 最终赚到的钱。
- $A^\pi(s, a)$: 就是你自己凭借自己判断买茅台比随便选股票多赚到的钱。

所以当 $A^\pi(s, a) > 0$: 这个动作比平均水平好, 应该增加其概率, 反之则减少。

12.3.2. Vanilla Policy Gradient(朴素策略梯度)

这是所有策略梯度算法的基础。它的核心思想非常简单:**如果一个动作带来了高回报, 就通过梯度上升增加该动作的概率。**

核心定义与符号

- **轨迹**: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ 。
- **轨迹回报**: $R(\tau) = \sum_{t=0}^T \gamma^t r_t$ 。
- **目标函数**: 最大化期望回报 $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$ 。
- $\theta \in \mathbb{R}^n$: 模型的参数向量 (例如神经网络的权重)。
- **x**: 随机变量 (在 RL 中对应状态-动作对 (s, a))。

数学原理推导

我们的目标是**计算梯度** $\nabla_\theta J(\theta)$ 。 $J(\theta) = \int P(\tau|\theta) R(\tau) d\tau$

Keynote 12.1

因为 $R(\tau)$ 奖励值是确定好的, 然后 $P(\tau|\theta) = P(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)$ 是和环境交互的反馈, 我们能调整的也就是策略函数 $\pi_\theta(a|s)$ 这一项。更具体一点, 也就是修改网络权重参数 θ , 使得 $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$, 所以求梯度 $\nabla_\theta J(\theta)$ 就是必经之路。

其中 $P(\tau|\theta)$ 是轨迹发生的概率。对 θ 求导: $\nabla_\theta J(\theta) = \int \nabla_\theta P(\tau|\theta) R(\tau) d\tau$ 这里运用著名的 **Log-Derivative Trick (对数导数技巧)**:

$\nabla \log x = \frac{\nabla x}{x}$, 所以 $\nabla x = x \nabla \log x$ 。(说白了也就是链式法则移个项)

代入上式有: $\nabla_\theta J(\theta) = \int P(\tau|\theta) \nabla_\theta \log P(\tau|\theta) R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log P(\tau|\theta) R(\tau)]$

展开轨迹概率的对数 (状态转移概率与 θ 无关, 求导为 0, 只剩下策略项): $\nabla_\theta \log P(\tau|\theta) = \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t)$

最终得到 **Vanilla PG 的梯度公式**: $\nabla_\theta J(\theta) = \mathbb{E}_t [\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A_t]$

注: 实际中通常用优势函数 A_t 替换回报 $R(\tau)$ 以降低方差

直观理解

- **直观理解**: $\nabla_\theta \log \pi_\theta$ 表示参数梯度方向, A_t 是标量权重, 和 α 共同组成步长大小。如果 A_t 很大 (表示当前的动作比平均动作带来的收益要好), 梯度就很大, 参数就大幅更新以增加该动作概率。
- **致命缺陷**:
 1. **步长 α 难以确定**: 步长太小, 训练极慢; 步长太大, 一次糟糕的更新会让策略参数 θ 飞到一个极差的区域。由于数据是根据当前策略采样的, 策略变差后采样的数据更差, 导致**无法恢复**。
 2. **采样效率低**: 由于是**在线的 (On-Policy)**, 每次更新完策略, 旧数据就作废了。

Vanilla PG

2026年1月9日 星期五 19:32

RL 简单回顾

 S_t 表示 t 时刻状态 a_t 表示 t 时刻动作 $\pi_\theta(a|s)$ 表示由参数 θ 控制的神经网络在 s 情况下采取 a 的概率 $A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$ 表示动作比平均水平强多少。 $A_\pi(s, a)$ 优势函数。Vanilla PG 属于直接优化策略参数的算法。 τ (轨迹) 表示 agent 与 env 交互的完整序列 $\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$ $P(\tau|\theta)$: 在策略各 π_θ 下, 产生轨迹 τ 的概率

$$\begin{aligned} \text{优化目标: } J(\theta) &= E_{\tau \sim \pi_\theta} [R(\tau)] \\ &= \sum_{\tau} P(\tau|\theta) R(\tau) \\ &= \int P(\tau|\theta) R(\tau) d\tau \end{aligned}$$

由于奖励 $R(\tau)$ - 一般是事先约定好的, 我们优化的点就在 $P(\tau|\theta)$ 上。

$$P(\tau|\theta) = \underbrace{P(s_0)}_{\text{初始状态}} \prod_{t=0}^{T-1} \underbrace{\pi_\theta(a_t|s_t)}_{\text{策略函数}} \underbrace{P(s_{t+1}|s_t, a_t)}_{\text{环境交互反馈}}$$

我们主要调整的, 也就是 θ (网络参数), 希望得到一条回报最高的轨迹。

$$\begin{aligned} \nabla_\theta J(\theta) &= \int \nabla_\theta P(\tau|\theta) R(\tau) d\tau \quad \text{log-derivative 技巧 (也就是链式法则)} \\ &= \int P(\tau|\theta) \nabla_\theta \log P(\tau|\theta) \cdot R(\tau) d\tau. \quad \frac{d \log f(\theta)}{d\theta} = \frac{1}{f(\theta)} \cdot \frac{df(\theta)}{d\theta} \Rightarrow f(\theta) \cdot \nabla_\theta f(\theta) = \frac{df(\theta)}{d\theta} \\ &= E_{\tau \sim \pi_\theta} [\nabla_\theta \log P(\tau|\theta) R(\tau)] \end{aligned}$$

$$\begin{aligned} \nabla_\theta \log P(\tau|\theta) &= \nabla_\theta \log [P(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)] \\ &= \nabla_\theta \log P(s_0) + \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) + \sum_{t=0}^{T-1} \nabla_\theta \log P(s_{t+1}|s_t, a_t) \\ &= \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \end{aligned}$$

$$\text{最终结果: } \nabla_\theta J(\theta) = E_t [\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \hat{A}_t] \quad \begin{array}{l} \rightarrow \text{优势函数代替 } R(\tau) \\ \text{以降低方差.} \end{array}$$

图 12.2: Vanilla PG 的梯度公式

12.3.3. TRPO

TRPO 是为了解决 Vanilla PG 的步长问题而提出的。它的核心思想是“信任域”。

为了防止步子太大扯到蛋，用旧策略 π_{old} 收集的数据来评估改进方向。但这些数据只在新策略 π_{new} 和旧策略差异不大时才可信。那么该如何量化“差异不大”？即我们后文会详细展开的 **KL 散度**: $D_{KL}(\pi_{new} \parallel \pi_{old}) \leq \delta$

总结来说叫信任域是因为你信任旧策略的数据在这个范围内仍然有效，超出这个范围，数据就不可信了。

核心定义与符号

- **KL 散度**: $D_{KL}(\pi_{\theta_{old}}(\cdot|s) \parallel \pi_{\theta}(\cdot|s))$, 衡量两个概率分布的距离。

$D_{KL}(P \parallel Q) = \int_{-\infty}^{\infty} p(x) \log \frac{p(x)}{q(x)} dx = P(x) \log \frac{P(x)}{q(x)}$, 具体知识比如和交叉熵的关系请参考之前的笔记

- **信任区域 (Trust Region)**: 我们只相信在以旧策略为中心的一个小球内的更新。

数学原理推导

TRPO 不再直接指定学习率 α , 而是把优化问题转化为一个带约束的优化问题。

我们希望最大化目标函数, 但限制新旧策略之间的 KL 散度不超过 δ :

$$\begin{aligned} \max_{\theta} \quad & \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t \right] \\ \text{subject to} \quad & \mathbb{E}_t [D_{KL}(\pi_{\theta_{old}}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t))] \leq \delta \end{aligned}$$

Extension 12.1 为什么约束的是策略, 而不是参数, 我们不是要改进的就是参数吗?

因为神经网络的参数和输出行为是非线性关系, 你约束了 θ , 但最后输出的策略可能还是会超出我们接受的范围。直接约束行为变化, 而不是参数变化, 那么无论你网络结构怎么变、参数怎么编码, 只要行为 (输出概率) 变化在可接受范围内, 我们就认为参数变化在可接受范围内。

由于这个约束优化难以直接求解。为了求解它, TRPO 做了以下近似:

1. **一阶泰勒展开**: 将目标函数进行一阶展开。
2. **二阶泰勒展开**: 将 KL 散度约束项进行二阶展开, 近似为 $\frac{1}{2}(\theta - \theta_{old})^T H(\theta - \theta_{old})$, 其中 H 是 **费雪信息矩阵** (具体过程详见下文)。

证明一阶泰勒展开

1. **新策略比旧策略额外的奖励回报**:

$$J(\theta) - J(\theta_{old}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{old}}(s_t, a_t) \right]$$

$$\text{把奖励展开为状态和策略 } J(\theta) - J(\theta_{old}) = \sum_s \rho_{\pi_{\theta}}(s) \sum_a \pi_{\theta}(a|s) A^{\pi_{old}}(s, a)$$

2. 定义修改:

TRPO 做了一个近似:假设在局部更新时,状态分布的变化忽略不计,即 $\rho_{\pi_{\theta}}(s) \approx$

$$\rho_{\pi_{old}}(s)$$

于是我们定义替代目标函数 $L(\theta)$:

$$L(\theta) = J(\theta_{old}) + \sum_s \rho_{\pi_{old}}(s) \sum_a \pi_{\theta}(a|s) A^{\pi_{old}}(s, a)$$

利用重要性采样

$$\sum_a \pi_{\theta}(a|s) A(s, a) = \sum_a \frac{\pi_{old}(a|s)}{\pi_{old}(a|s)} \cdot \pi_{\theta}(a|s) A(s, a) = \sum_a \pi_{old}(a|s) \cdot \left[\frac{\pi_{\theta}(a|s)}{\pi_{old}(a|s)} A(s, a) \right] =$$

$$\mathbb{E}_{a \sim \pi_{old}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{old}(a|s)} A(s, a) \right]$$

$$\text{可得 } L(\theta) = J(\theta_{old}) + \mathbb{E}_{s \sim \rho_{old}, a \sim \pi_{old}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{old}(a|s)} A^{\pi_{old}}(s, a) \right]$$

3. 一阶泰勒展开:

现在,我们要对 $L(\theta)$ 在 θ_{old} 处进行一阶泰勒展开。 $L(\theta) \approx L(\theta_{old}) + \nabla_{\theta} L(\theta) \Big|_{\theta=\theta_{old}} (\theta - \theta_{old})$

- 计算常数项 $L(\theta_{old})$, 当 $\theta = \theta_{old}$ 时, 概率比率为 1。 $L(\theta_{old}) = J(\theta_{old}) + \mathbb{E}_{s, a \sim \pi_{old}} [1 \cdot A^{\pi_{old}}(s, a)]$

根据优势函数的定义, 其在自身策略下的期望为 0 (即 $\mathbb{E}[Q - V] = V - V = 0$)。 $\mathbb{E}_{a \sim \pi_{old}} [A^{\pi_{old}}(s, a)] = 0$

所以: $L(\theta_{old}) = J(\theta_{old})$

- 计算梯度项 $\nabla_{\theta} L(\theta) \Big|_{\theta=\theta_{old}}$: 这正是我们在上一个问题中证明过的结论。

$$\nabla_{\theta} L(\theta) = \mathbb{E} \left[\frac{\nabla_{\theta} \pi_{\theta}}{\pi_{\theta_{old}}} A^{\pi_{old}} \right]$$

在 $\theta = \theta_{old}$ 处: $\nabla_{\theta} L(\theta) \Big|_{\theta=\theta_{old}} = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a|s) \Big|_{\theta_{old}} \cdot A^{\pi_{old}}]$ (计算方式就是先用对数导数技巧换成导数乘 π , 然后消掉分母)

这正是 **Vanilla PG 的梯度**, 我们记为 $\mathbf{g}$ 。

4. 最终展开结果:

将上述两项代回泰勒公式:

$$L(\theta) \approx J(\theta_{old}) + \mathbf{g}^T (\theta - \theta_{old})$$

二阶泰勒展开与费雪信息矩阵

符号定义

- $\theta \in \mathbb{R}^n$: 模型的参数向量。
- $\mathbf{x}$: 随机变量。
- $p(x|\theta)$ 或 $\pi_{\theta}(a|s)$: 由参数 θ 决定的概率分布 (即策略)。
- $\mathcal{L}(\theta) = \log p(x|\theta)$: 对数似然函数。
- $\mathbf{g} = \nabla_{\theta} \mathcal{L}(\theta)$: 分数函数, 即对数似然的梯度, 实际上就是在欧几里何空间下的梯度。
- $\mathbf{F}(\theta)$: 费雪信息矩阵。

直观理解

KL散度的Taylor展开

2026年1月13日 星期二 17:47

分析参数从 θ_{old} 到 $\theta = \theta_{old} + \Delta\theta$ 时 KL 散度的变化.

$$\text{令 } f(\theta) = D_{KL}(\pi_{\theta_{old}} \parallel \pi_{\theta}) = \mathbb{E}_{x \sim \pi_{\theta_{old}}} [\log P(\pi_{\theta_{old}}) - \log P(\pi_{\theta})]$$

在 θ_{old} 处对 $f(\theta)$ Taylor 展开 (= 所)

$$f(\theta) \approx f(\theta_{old}) + (\nabla_{\theta} f(\theta_{old}))^T \Delta\theta + \frac{1}{2} \Delta\theta^T \cdot (\nabla_{\theta}^2 f(\theta_{old})) \Delta\theta$$

这里有小细节, 因为最终形状会和 $f(\theta)$ 一致, 而不是 θ .
(假设为1)

这种情况肯定不能把 θ 向量放矩阵乘法前面, 而是把 θ^T 放前面. 举个例子, $\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}_{2 \times 1}$

$$\text{那么 } \theta^T = \begin{bmatrix} \theta_1^2 & \theta_1 \theta_2 \\ \theta_2 \theta_1 & \theta_2^2 \end{bmatrix}$$

$$\theta^T \theta = \theta_1^2 + \theta_2^2$$

对于 $f(\theta_{old}) = D_{KL}(\theta_{old} \parallel \theta_{old}) = 0$. (前后一致)

第=项. 同理 $f(\theta_{old}) = 0 \Rightarrow (\nabla_{\theta} f(\theta_{old}))^T \Delta\theta = 0$.

此时只剩第三项: $f(\theta) = \frac{1}{2} \Delta\theta^T (\nabla_{\theta}^2 f(\theta_{old})) \Delta\theta$

$$\text{- 所导: } \nabla_{\theta} f(\theta_{old}) = -\mathbb{E}_{x \sim \pi_{\theta_{old}}} [\nabla_{\theta} \log \pi_{\theta}(x)]$$

$$\text{- 所导: } \nabla_{\theta}^2 f(\theta_{old}) = -\mathbb{E}_{x \sim \pi_{\theta_{old}}} [\nabla_{\theta}^2 \log \pi_{\theta}(x)] = F(\theta) \text{ 费雪信息矩阵.}$$

$$\text{因此, } f(\theta) \approx \frac{1}{2} \Delta\theta^T F(\theta) \Delta\theta$$

图 12.3: KL 散度的二阶泰勒展开

用来**衡量观测数据中包含了多少关于模型参数的信息**

假设你在估计一个参数 θ :

- **如果参数稍微变一点, 模型的概率分布就变化很大**: 数据对参数很敏感 -> 费雪信息大
- **如果参数变了, 模型几乎没变化**: 数据难以区分不同参数 -> 费雪信息小
- **所以: 费雪信息越大, 参数越容易被准确估计**

费雪信息矩阵的定义

$$F(\theta) = \mathbb{E} [\nabla_{\theta} \log p(X | \theta) \nabla_{\theta} \log p(X | \theta)^{\top}]$$

其中:

- $\nabla_{\theta} \log p(X | \theta)$: 对数似然的梯度 (也被称为得分函数)
- 期望是对 $X \sim p(x|\theta)$ 取的

等价形式 (常用)

$$\text{在满足一定正则条件下: } F(\theta) = -\mathbb{E} [\nabla_{\theta}^2 \log p(X | \theta)]$$

也就是说:

费雪信息 = 对数似然 Hessian 的负期望

具体推导过程:

Extension 12.2 为什么费雪信息矩阵要这样定义?

对数似然梯度反映了一种**瞬时敏感度**, $\nabla_{\theta} \log p(x|\theta)$ 表示:**在观测到样本 x 的情况下, 参数 θ 的微小变化会让概率变化多快**。结果大 $\rightarrow$ 这个样本对参数很敏感, 结果小 $\rightarrow$ 这个样本几乎没提供区分能力

Extension 12.3 为什么是外积形式 (而不是 Hessian)?

一方面在正则条件下, 两者**严格相等**:

$$F(\theta) = -\mathbb{E} [\nabla_{\theta}^2 \log p(X|\theta)]$$

但外积形式有几个深层优势:

1. **必然正半定 (信息不能为负)**
2. 与 KL / 信息几何自然一致

Extension 12.4 为什么对约束的 KL 散度是二阶展开, 对目标函数就只是一阶泰勒展开?

1. **KL 在最优点的一阶项是 0 (这是本质差异)**

KL 约束项:

$$\bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) = \mathbb{E}_s [D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_{\theta})]$$

在 $\theta = \theta_{\text{old}}$ 处:

- KL = 0

- 一阶导数 = 0 (说明这是**局部最小值**)

因此:

费雪信息矩阵.

$$F(\theta) = E_{x \sim p(x|\theta)} [\nabla_{\theta} \log p(x|\theta) \nabla_{\theta} \log p(x|\theta)^T]$$

对于得分函数. $\log p(x|\theta)$.

- 一阶导:

$$\nabla_{\theta} \log p(x|\theta) = \frac{\nabla_{\theta} p(x|\theta)}{p(x|\theta)}$$

反带回去.

- 二阶导:

$$\begin{aligned} \nabla_{\theta}^2 \log p(x|\theta) &= \nabla_{\theta} \left(\frac{\nabla_{\theta} p(x|\theta)}{p(x|\theta)} \right) \\ &= \frac{\nabla_{\theta}^2 p(x|\theta) \cdot p(x|\theta) - (\nabla_{\theta} p(x|\theta))^2}{p^2(x|\theta)} \end{aligned}$$

$$\left(\frac{u}{v} \right)' = \frac{u'v - uv'}{v^2}$$

这里是矩阵, 要写成 $\nabla_{\theta} p(x|\theta) \cdot (\nabla_{\theta} p(x|\theta))^T$

$$= \frac{\nabla_{\theta}^2 p(x|\theta)}{p(x|\theta)} - \left(\frac{\nabla_{\theta} p(x|\theta)}{p(x|\theta)} \right) \left(\frac{\nabla_{\theta} p(x|\theta)}{p(x|\theta)} \right)^T$$

$$= \frac{\nabla_{\theta}^2 p(x|\theta)}{p(x|\theta)} - \nabla_{\theta} \log p(x|\theta) (\nabla_{\theta} \log p(x|\theta))^T$$

可以看到这已经是费雪矩阵的形式了.

求期望:

$$E_{x \sim p(x|\theta)} [\nabla_{\theta}^2 \log p(x|\theta)] = E \left[\frac{\nabla_{\theta}^2 p(x|\theta)}{p(x|\theta)} \right] - F(\theta)$$

展开

$$= \int \left[\frac{\nabla_{\theta}^2 p(x|\theta)}{p(x|\theta)} \right] p(x|\theta) dx$$

$$= \int \nabla_{\theta}^2 p(x|\theta) dx$$

$$= \nabla_{\theta}^2 \int p(x|\theta) dx \quad (\text{概率和为1})$$

$$= \nabla_{\theta}^2 1 = 0.$$

$$\Rightarrow F(\theta) = - E_{x \sim p(x|\theta)} [\nabla_{\theta}^2 \log p(x|\theta)]$$

图 12.4: 费雪信息矩阵与海斯矩阵的关系

$$\bar{D}_{KL}(\theta_{old}, \theta) = \frac{1}{2} \Delta \theta^T H_{KL} \Delta \theta + o(\|\Delta \theta\|^2)$$

如果你只做一阶近似, $KL \approx 0$, 约束直接消失, 没有意义。

所以:

- **目标函数**: 一阶就有非零信息 (梯度)
- **KL 约束**: 一阶项恒为 0, 必须保留二阶

2. 为什么不对目标也做二阶泰勒展开?

数学上虽然能做到, 但是:

二阶泰勒展开后 H_L 是目标函数的 Hessian 矩阵:

$$H_L = \nabla_{\theta}^2 L(\theta) = \mathbb{E} [\nabla_{\theta}^2 \log \pi_{\theta}(a|s) \cdot A + \nabla_{\theta} \log \pi_{\theta} \nabla_{\theta} \log \pi_{\theta}^T \cdot A]$$

- 目标 Hessian 矩阵: - **不一定正定**, 这受到优势函数 A 的影响。我们希望**最大化 $L(\theta)$** , 如果二阶项是负的, 就处于局部最大值, 优化器会认为”离得越远越好”, 这在数值上非常不稳定。- KL 散度的 Hessian 矩阵只和策略函数 π 有关, 而 H_L 还和优势函数 A 有关, 每个样本的 A 都各不相同, **方差太大导致噪声极大**。

所以 TRPO 的设计选择是:

用 KL 的二阶结构控制 “走多远” (步长), 用目标的一阶信息决定 “往哪走” (梯度方向)。

二次规划近似形式

经过两次近似后, TRPO 可以近似成一个**标准二次规划**(忽略常数项):

$$\max_{\Delta \theta} \quad g^T \Delta \theta$$

$$\text{s.t.} \quad \frac{1}{2} \Delta \theta^T F \Delta \theta \leq \delta$$

解这个二次规划: 得到自然梯度方向

这是一个经典结果: 在线性目标、椭球约束下, 最优解方向是 $F^{-1}g$ 。

推导:

构造**拉格朗日函数**:

$$\mathcal{L}(\Delta \theta, \lambda) = g^T \Delta \theta - \lambda \left(\frac{1}{2} \Delta \theta^T F \Delta \theta - \delta \right)$$

对 $\Delta \theta$ 求导并置零:

$$\nabla_{\Delta \theta} \mathcal{L} = g - \lambda F \Delta \theta = 0 \Rightarrow \Delta \theta = \frac{1}{\lambda} F^{-1} g$$

再用约束饱和确定缩放因子:

$$\frac{1}{2} \Delta \theta^T F \Delta \theta = \frac{1}{2} \frac{1}{\lambda^2} g^T F^{-1} g = \delta \Rightarrow \lambda = \sqrt{\frac{g^T F^{-1} g}{2\delta}}$$

$$\text{所以 } \Delta \theta^* = \sqrt{\frac{2\delta}{g^T F^{-1} g}} F^{-1} g$$

Keynote 12.2

实际 TRPO 算法在计算出这个更新方向后, 会进行**Line Search(线性搜索)**。即, 先尝试走完这步, 检查: KL 散度是否真的 $\leq \delta$? 目标函数 $L(\theta)$ 是否真的提升了? 如果不满足, 就按比例缩小步长 (如 $\times 0.5, \times 0.25 \dots$) 直到满足为止。这是 “Trust Region” 保证单调不下降的关键工程实现。

方向 $F^{-1}g$ 就是自然梯度方向。这导出了自然梯度的更新方向: $\theta_{new} = \theta_{old} + \beta F^{-1}g$

其中 F^{-1} 校正了欧几里得空间的梯度方向, 使其符合黎曼几何 (概率流形) 的曲率。

黎曼几何空间与自然梯度的简要说明

在普通梯度下降中, 我们更新参数的规则是: $\theta_{new} = \theta_{old} + \alpha \nabla_{\theta} J(\theta)$

这隐含了一个假设: 我们在参数空间 Θ 中移动一个固定的欧几里得距离 $\|\Delta\theta\|^2$, 会导致策略表现发生可预期的变化。但实际上, 参数空间的小微扰 $\Delta\theta$ 可能导致输出概率分布 π_{θ} 发生剧烈变化, 也可能几乎不变。参数空间是平坦的, 但策略空间是弯曲的。所以, 我们之所以引入黎曼几何是因为距离的定义和位置相关。在概率单纯形上, 衡量两个策略 π_{θ} 和 $\pi_{\theta+\delta\theta}$ 差异的最自然度量不是 $\|\delta\theta\|^2$, 而是二阶 KL 散度, 也就是我们之前推导的费雪信息矩阵 $F(\theta)$ 。

$F(\theta)$ 扮演了度量张量的角色。它定义了流形上的局部曲率。 F^{-1} 就是对欧几里得空间的梯度 g 进行了“矫正”, 使其适应流形的曲率。 $F^{-1}\nabla_{\theta}J$ 或 $F^{-1}g$ 就是黎曼几何空间下的梯度, 也被称为自然梯度。

总结

TRPO 的核心想法是“信任域”, 也就是基于旧策略数据优化而来的新策略不可以与原来的旧策略差的太远。具体的优化过程基于黎曼几何空间的梯度下降, 使用了标准梯度 g 乘上矫正的费雪信息矩阵 F^{-1} 也就是自然梯度方向。

工业界落地的缺陷

- **计算灾难**: F 是一个 $|\theta| \times |\theta|$ 的矩阵。如果网络有 100 万个参数, 计算 F 的逆矩阵 F^{-1} 需要 $O(N^3)$ 的复杂度, 太高了。
- **共轭梯度法**: 虽然 TRPO 使用共轭梯度法 (一种迭代的计算方法) 来近似计算 $H^{-1}g$, 避免了直接求逆, 但实现起来依然非常复杂且慢。

12.4. GAE

GAE (Generalized Advantage Estimation) 是现代强化学习 (尤其是 Actor-Critic 架构) 中处理 **偏差-方差权衡**, 它的核心贡献在于它提供了一种数学上优雅的方法, 通过调节参数 λ , 在无偏的高方差估计 (Monte-Carlo) 和有偏的低方差估计 (TD) 之间找到 trade-off 平衡。

12.4.1. 省流版

在 PPO 这种基于策略的 on-policy 算法里, 我们采样的最终结果或者说是样本是一条完整的轨迹及其奖励/回报, 我们知道优势函数 A_t 的定义是 $A_t = Q^\pi(s, a) - V^\pi(s)$, 用样本去估计的结果就是 $A_t = G_t - V^\pi(s)$, G_t 就表示实际采样的结果, 那么这个缺陷就比较明显, 因为即使是同一策略, 不同的轨迹样本之间的差别也会很大, 这就导致了方差变大。因此 GAE 的想法是从 TD 误差上做文章, TD 误差简单说就是把 $Q^\pi(s, a)$ 表示成 $r(s, a, s') + \gamma V^\pi(s')$ 这种开展一步或两步或 n 步的形式, 这样方差就不像轨迹那样大了, 因为我只关注某几步小变化。具体来讲, GAE 则把同一条轨迹的信息拆成多尺度的 TD 信号, 并用 λ 平滑融合, 从而得到好的优势函数 A_t

12.4.2. 核心定义与符号体系

- $\lambda \in [0, 1]$: GAE 平滑参数, 控制偏差与方差的权衡。
- $\hat{A}_t^{\text{GAE}}$: 广义优势估计量。
- δ_t^V : TD 误差。

12.4.3. 问题背景

$$A_\pi(s_t, a_t) = \mathbb{E} s_t + 1 [r_t + \gamma V_\pi(s_{t+1})] - V_\pi(s_t)$$

但在实际训练中, 我们无法获知真实的 V_π (只能用神经网络近似 V_ϕ), 也不知道未来的确切奖励。因此我们需要 **估计优势函数**。

12.4.4. 数学原理推导

Monte Carlo Advantage

$$A_t = G_t - V(s_t)$$

- 用轨迹的完整回报 G_t
- 无偏 (因为毕竟是真实值), 但由于是轨迹而不是某几个点, 导致方差较大

TD Error (δ)

我们定义 V_ϕ 下的 TD Error 为 δ_t^V 。这是最基础的单步优势估计: $\delta_t^V = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$

Keynote 12.3

注意: 这里 δ_t^V 其实就是 $A(s_t, a_t)$ 的一个 **有偏估计** (因为 V_ϕ 不是真实的 V_π), 但它的方差很低, 因为它只依赖一步真实的 r_t 。

展开多步估计

我们可以构造一系列的优势估计量 $A_t^{(k)}$, 代表向前看 k 步:

- **1-step (k=1):** $\hat{A}_t^{(1)} = \delta_t^V = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
- **2-step (k=2):** $\hat{A}_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V_\phi(s_{t+2}) - V_\phi(s_t)$
- **k-step:** $\hat{A}_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V_\phi(s_t) + \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k V_\phi(s_{t+k})$
- **∞ -step (其实就等价 Monte Carlo):** $\hat{A}_t^{(\infty)} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l}^V = (\sum_{l=0}^{\infty} \gamma^l r_{t+l}) - V_\phi(s_t)$
(低偏差, 高方差 - 因为累积了每一步环境的随机性)

GAE 的核心: 指数加权平均

GAE 的优点在于, 他不选择特定的 k , 而是将所有可能的 k -step 估计量进行**指数加权平均**。权重由参数 λ 控制。

定义 GAE 为: $\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = (1 - \lambda) \left(\hat{A}_t^{(1)} + \lambda \hat{A}_t^{(2)} + \lambda^2 \hat{A}_t^{(3)} + \dots \right)$

这是一个几何级数求和。推导后的最终形态:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V$$

工程递归计算公式

在代码实现中, 我们不会计算无穷级数, 而是使用递归形式, 从轨迹的最后一步向前计算:

$$\hat{A}_t^{\text{GAE}} = \delta_t^V + (\gamma \lambda) \hat{A}_{t+1}^{\text{GAE}}$$

其中 $\delta_t^V = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ 。

采样的轨迹会提供一系列奖励 r , 然后 $V(s)$ 可以通过神经网络得到。

12.4.5. 直觉理解

λ 作为 “置信度滑块”

你可以把 λ 想象成一个调节我们对 “**Critic 网络预测能力 (V_ϕ)**” vs “**现实世界反馈 (r_t)**” 信任程度的滑块。

- **当 $\lambda = 0$ (TD):** $\hat{A}_t = \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
 - **物理意义:** 目光短浅。我们只相信当前这一步的奖励 r_t , 对于未来, 我们完全依赖 Critic 的预测 $V_\phi(s_{t+1})$ 。
 - **后果:** 方差最小 (只引入了一步随机性), 但偏差最大 (如果 Critic 没训练好, 估计就全错了)。
- **当 $\lambda = 1$ (Monte Carlo):** $\hat{A}_t = \sum \gamma^l \delta_{t+l} = \text{Return} - V_\phi(s_t)$
 - **物理意义:** 实事求是。我们完全不信任 Critic 对未来的预测, 我们一直等到 episode 结束, 把所有实际拿到的奖励加起来。
 - **后果:** 偏差最小 (基于真实数据), 但方差最大 (环境中的每一点风吹草动都会累积到回报中, 导致训练震荡)。

Monte Carlo 下的 A_t .

$$A_t = G_t - V(S_t)$$

其中 G_t 表示在 S_t, a_t 下的后续完整轨迹.

TD 误差下的 A_t .

$$A_t = Q(S_t, a_t) - V(S_t)$$

$$= r_t + \alpha V(S_{t+1}) - V(S_t)$$

→ 为了便于区分, 用 α 代替 γ 表示折扣系数.

令 G_t 表示动作价值函数 $Q(S_t, a_t)$ 的拆分回报.

$$\text{一步回报 } G_t^{(1)} = r_t + \alpha V(S_{t+1})$$

$$\text{二步: } G_t^{(2)} = r_t + \alpha r_{t+1} + \alpha^2 V(S_{t+2})$$

:

$$n \text{ 步: } G_t^{(n)} = \sum_{k=0}^{n-1} \alpha^k \cdot r_{t+k} + \alpha^n V(S_{t+n})$$

$$\text{对应 } n \text{ 步下的 } A_t = G_t^{(n)} - V(S_t)$$

令 TD 单步误差为 δ

$$\text{则 } \delta_t = r_t + \alpha V_{t+1} - V_t$$

$$\delta_{t+1} = r_{t+1} + \alpha V_{t+2} - V_{t+1}$$

$$\text{①} + \alpha \text{ ② (为了消去 } V_{t+1})$$

$$\rightarrow \delta_t + \alpha \delta_{t+1} = r_t + \alpha r_{t+1} + \alpha^2 V_{t+2} - V_t$$

$$= G_t^{(2)} - V_t$$

$$\text{因此可得: } A_t^{(2)} = G_t^{(2)} - V_t$$

$$= \delta_t + \alpha \delta_{t+1}$$

$$\text{同理: } A_t^{(n)} = \sum_{k=0}^{n-1} \alpha^k \delta_{t+k} \quad \text{这说明 } n \text{ 步误差函数 } A = \text{TD 误差与折扣系数加权和}$$

对于 GAE:

$$A_t^{\text{GAE}} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} A_t^{(n)}$$

用 λ 对 A_t 做了一个加权平均

$$= (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{k=0}^{n-1} \alpha^k \delta_{t+k}$$

→ 通过这里能看出来 $n=1$ 时, 只存在 $k=0$.
 $n=2$ 时, 存在 $k=1, 0$.

也就是 $n \geq k+1$ 范围存活.

对于 δ_{t+k} 这一项而言, 它的系数可以写作 $(1-\lambda) \sum_{n=k+1}^{\infty} \lambda^{n-1} \alpha^k$

$$= (1-\lambda) \alpha^k \sum_{n=k+1}^{\infty} \lambda^{n-1}$$

$$= (1-\lambda) \alpha^k \frac{\lambda^k}{1-\lambda}$$

$$= (\alpha \lambda)^k$$

$$\therefore A_t^{\text{GAE}} = \sum_{k=0}^{\infty} (\alpha \lambda)^k \cdot \delta_{t+k}$$

图 12.5: GAE

- 当 $0 < \lambda < 1$ (GAE): 我们构建了一个**衰减窗口**。我们利用近期的真实奖励来修正 Critic 的预测, 同时利用 Critic 的预测来平滑远期的随机性。

信号处理视角的“滤波器”

假设我们有一个真实的累积回报信号 G_t 。我们可以将其分解为两部分: $G_t =$

$$\underbrace{V_{\pi}(s_t)}_{\text{均值/趋势}} + \underbrace{A_{\pi}(s_t, a_t)}_{\text{波动/细节}} + \underbrace{\epsilon}_{\text{噪声}}$$

1. 低频信息 $\approx$ 价值函数 $V(s)$ 的偏差

- **定义:** 这是信号中的“直流分量”或“缓慢变化的趋势”。
- **RL 对应:** 由 Critic 网络 $V_{\phi}(s)$ 提供的预测值。
- **特性:** 因为它是一个神经网络的输出, 它是对无数条历史轨迹的平均。因此它是**平滑的**、**稳定的**, 但在训练初期通常是不准确的 (存在系统性偏差)。

2. 高频信息 $\approx$ 蒙特卡洛采样的方差

- **定义:** 这是信号中快速跳变、剧烈震荡的部分。
- **RL 对应:** 单次采样轨迹中具体的奖励 r_t 和状态跳转。
- **特性:** 如果你把同一个策略运行 100 次, 每一次的轨迹 (高频细节) 都不同。这不仅包含你的动作带来的真实反馈 (有效高频), 也包含环境掷骰子的随机性 (无效高频噪声)。

回顾 GAE 的定义: $\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$

这就好比我们在时间轴上对 TD Error 序列 δ 做了一个**卷积**。 $\hat{A} = \delta * \text{Kernel}_{\lambda}$

其中卷积核是指数衰减函数 $f(k) = (\gamma\lambda)^k f$

低频与高频

让我们看看 λ 如何控制这个“滤波器”的通带:

1. 当 $\lambda \rightarrow 0$ (低通滤波器 / 截止频率极低)

- 卷积核迅速衰减, 只保留第一项 δ_t 。
- 公式退化为: $r_t + \gamma V(s_{t+1}) - V(s_t)$ 。
- 频域解释: 我们切断了未来的所有高频波动。我们只看这一步的 r_t , 剩下的全部用 $V(s)$ 这个“低频均值”来代替。
- 结果: 滤除了蒙特卡洛带来的高频噪声 (方差极低), 但引入了 $V(s)$ 自身的低频系统性错误 (偏差大)。我们过于信任“趋势”。

2. 当 $\lambda \rightarrow 1$ (全通滤波器)

- 卷积核衰减很慢, 累加了长远的未来。
- 公式退化为: $R_t - V(s_t)$ 。

- 频域解释: 我们允许所有的频率通过。哪怕 100 步之后发生了一个随机事件导致奖励剧变, 这个震荡 (高频信号) 也会完整地传导回 t 时刻。
- 结果: 消除了 Critic 的系统性偏差 (因为最终用的是真实回报), 但引入了巨大的高频环境噪声 (方差极大)。我们过于信任“细节”。

12.5. PPO

从本质上讲,TRPO 通过在参数空间施加“**硬约束**”(即 **KL 散度约束**)来保证策略更新的**单调性**,这是一个优雅的二阶优化问题。然而,在工程实践中,计算 Hessian 矩阵的逆(或使用共轭梯度法)代价极其昂贵。

PPO 的核心使命,就是用简化的方法,去逼近 TRPO 的二阶性能。也就是把 TRPO 的“硬约束”转化为目标函数中的“**截断机制**”。

12.5.1. 核心定义与符号体系

- $\pi_{\theta}(a|s)$: 当前待优化的策略网络, 参数为 θ 。
- $\pi_{\theta_{\text{old}}}(a|s)$: 在本次更新迭代前的旧策略, 用于采样数据。
- $\hat{A}_t$: 在时间步 t 的优势函数估计值。通常使用 GAE 计算。
- $r_t(\theta)$: **概率比率**, 这是 PPO 的核心变量: $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ 注意: 当 $\theta = \theta_{\text{old}}$ 时, $r_t(\theta) = 1$ 。
- ϵ : 超参数, 截断范围, 通常取值 0.1 或 0.2。
- $J(\theta)$: 保守策略迭代目标函数, 即 TRPO 的无约束目标部分。

12.5.2. 数学原理推导

TRPO 简要回顾

TRPO 试图解决的问题是最大化以下目标函数, 同时满足 KL 散度约束: $\max_{\theta} \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right]$
s.t. $\hat{\mathbb{E}}_t [D_{KL}(\pi_{\theta_{\text{old}}}(\cdot|s_t) || \pi_{\theta}(\cdot|s_t))] \leq \delta$

直接对约束求解代价很高,PPO 是直接对比率进行**截断 clip**。

构建无约束代理目标

首先, 我们定义无约束的基础目标函数: $J(\theta) = \hat{\mathbb{E}}_t [r_t(\theta) \hat{A}_t]$

如果我们直接最大化这个公式(不加约束), 由于 $r_t(\theta)$ 可能变得非常大(新旧策略差异过大), 会导致策略更新步长过大, 从而发生策略坍塌。

引入截断机制

PPO 的核心创新在于修改了 $J(\theta)$ 。它强制要求 $r_t(\theta)$ 保持在区间 $[1 - \epsilon, 1 + \epsilon]$ 附近。

我们需要定义一个**截断后的比率**: $r_t^{\text{clip}}(\theta) = \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$

此时,PPO-Clip 的目标函数构造为: $J^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] - \beta D_{KL}(\pi_{\theta} || \pi_{\text{ref}})$

其中第二项表示最新的模型不可以比参考模型偏的太远, β 是调节的参数, 现在比较流行使用**自适应的方式**调节 β

$$\beta_{t+1} = \begin{cases} \beta_t \cdot \alpha & D_{KL} > \text{target} \\ \beta_t / \alpha & D_{KL} < \text{target} \end{cases}$$

这表示如果上一轮 $D_{KL} > \text{target}$ 即实际 KL 比预期大, 说明, 刚刚这轮更新, 策略变化太大, 因此增大 β , 加强这一项的惩罚。如果上一轮 $D_{KL} < \text{target}$ 即实际 KL 比预期小, 说明策略变化不大, 可以减少一下 β , 松弛一下惩罚。

最小值情况拆开考虑

1. $\hat{A}_t > 0$ (该动作优于平均, 应增加概率)

- 我们希望增加 $r_t(\theta)$ (即 $\pi_\theta > \pi_{\theta_{\text{old}}}$)。
- 目标函数变为: $\min(r_t(\theta)\hat{A}_t, (1 + \epsilon)\hat{A}_t)$ 。
- **数学含义**: 如果 $r_t(\theta)$ 增长超过 $1 + \epsilon$, 目标函数就锁定在 $(1 + \epsilon)\hat{A}_t$ 。这意味着**过大的更新不会带来额外的奖励, 从而消除了让 θ 剧烈变化的梯度动力**。

最终完整损失函数

实际训练的损失函数和目标函数有些不同, 我们还需要加入**价值函数损失 L^{VF} (用于更新 Critic)**和**熵正则项 S (用于鼓励探索)**:

$$L_t^{PPO}(\theta) = \hat{\mathbb{E}}_t [L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S\pi_\theta]$$

其中 c_1, c_2 是系数。

$$L_t^{VF}() = (V(s_t) - V_t^{\text{target}})^2$$

V_t^{target} 通常来自经过 GAE 操作后的样本

熵的定义

对离散策略 $\pi(a|s)$:

$$\mathcal{H}(\pi(\cdot|s)) = - \sum_a \pi(a|s) \log \pi(a|s)$$

熵这一项反映了不确定性, 其实就表示模型的探索程度, 防止策略过早收敛。

当策略变成确定性的时候, 熵变为 0, 也就没有探索了。当策略变成均匀分布, 也就是每种选择概率都一致时, 熵最大, 探索程度最高。

Extension 12.5 为什么 PPO 的损失函数和目标函数不一样呢?

$$\text{目标函数: } J^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t \right) \right] - \beta D_{KL}(\pi_\theta || \pi_{ref})$$

这是经过我们理论上得到的最优结果, 第一项是限制策略不比上一步偏差太多, 第二项是利用 KL 散度来限制模型不要比参考模型差太多, 实际上这个可以经过 **reward shaping** 变成一项。

$$J^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t \right) \right] - \beta \mathbb{E}[\log \frac{\pi_\theta}{\pi_{ref}}], \text{ 去掉期望 } \mathbb{E}, \text{ 实际上就变为了 } r = r_{clip} - \beta \log \frac{\pi_\theta}{\pi_{ref}} \text{ 成了一项。}$$

而实际在工程上, 为了更好训练神经网络, 我们要加一些辅助的 loss, 一方面要考虑到 Critic 网络, 因此补充上价值函数的损失; 另一方面我们不希望策略尽早收敛, 所以补充上了熵。

$$L_t^{PPO}(\theta) = \hat{\mathbb{E}}_t [L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S\pi_\theta]$$

标准的 PPO 实现里是把目标函数的 KL 散度以及 β 自适应策略放到外面, 如果 KL 偏差太大, 直接**早停**。但我查阅的一些工业界的代码里面, 也有把 KL 项放到损失函数里面的

$$Loss = -\hat{\mathbb{E}}_t [L_t^{CLIP}(\theta)] + c_1 L_t^{VF}(\theta) - c_2 S\pi_\theta + \beta D_{KL}$$

12.6. GRPO

不管是 PPO 还是 TRPO，它们都是基于 Actor-Critic 结构的，想要得到优势函数 $A^\pi(s, a) = Q^\pi(s, a) - V_\phi(s, a)$ ，需要训练一个 Critic 网络对状态价值函数 $V_\phi(s, a)$ 进行估计，而 GRPO 和 PPO 最大的差别就是它**放弃了 Critic 网络**，转而选择了**组内优化**。

12.6.1. 核心定义与符号体系

在开始推导之前，我们定义如下符号：

- Q : 问题的分布。
- q : 从分布中采样的问题。
- π_θ : 需要优化的策略网络，参数为 θ 。
- π_{ref} : 参考策略网络，通常是 SFT 之后的初始模型，用于约束 KL 散度。
- $\pi_{\theta_{old}}$: 在本次迭代更新前的旧策略网络。
- G : Group Size，即针对同一个问题 q ，我们采样的输出数量（例如 $G=64$ ）。
- $\{o_1, o_2, \dots, o_G\}$: 针对问题 q ，由 $\pi_{\theta_{old}}$ 采样生成的 G 个不同的输出。
- $r(o, q)$: 奖励函数，通常包含正确性奖励或过程奖励模型。
- A_i : 第 i 个输出的优势函数。
- ϵ : PPO 中的截断超参数。
- β : KL 散度惩罚系数。

12.6.2. 数学原理推导

GRPO 的核心构建：去 Critic 化

GRPO 放弃了 $V_\phi(s)$ 。它利用大模型生成的多样性，对同一个问题 q 生成一组输出 $\{o_1, \dots, o_G\}$ 。

GRPO 假设：**这 G 个输出的奖励分布，本身就可以作为基线 (Baseline)**。

对于第 i 个输出 o_i ，其对应的奖励为 $r_i = r(q, o_i)$ 。我们计算这组奖励的**平均值**和**标准差**： $\text{mean} = \bar{R} = \frac{1}{G} \sum_{i=1}^G r_i$

$$\text{std} = \sigma_R = \sqrt{\frac{1}{G} \sum_{i=1}^G (r_i - \bar{R})^2 + \delta}$$

δ 为数值稳定极小值

组内相对优势

GRPO 将优势函数定义为标准化后的奖励： $\hat{A}_i = \frac{r_i - \bar{R}}{\sigma_R}$

数学性质：这里 $\bar{R}$ 充当了动态 Baseline。如果 o_i 的得分高于组内平均分，则 $\hat{A}_i > 0$ ，模型会被鼓励增加生成该输出的概率；反之则被抑制。

目标函数

GRPO 将上述优势代入 PPO 的 Clip 损失函数中，并增加了 KL 散度惩罚项以防止**模型崩溃 (Reward Hacking)**。

最终的 GRPO 目标函数 $J_{GRPO}(\theta)$ 为: $J_{GRPO}(\theta) = \mathbb{E}_{q \sim Q, \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}} \left[\frac{1}{G} \sum_{i=1}^G \left(\mathcal{L}_{clip}^{(i)} - \beta \mathbb{D}_{KL}(\pi \right.$

其中, $\mathcal{L}_{clip}^{(i)}$ 是标准的 PPO 截断损失:

$$\mathcal{L}_{clip}^{(i)} = \min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} \hat{A}_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right)$$

这里的 KL 项通常采用近似计算:

$$\mathbb{D}_{KL} \approx \frac{\pi_{\theta_{old}}(o_i|q)}{\pi_{ref}(o_i|q)} - \log \frac{\pi_{\theta_{old}}(o_i|q)}{\pi_{ref}(o_i|q)} - 1$$

实际上主要区别还是在优势函数 A 的定义上, GRPO 目标函数的形式和 PPO 基本上一致。

Reward Hacking 的数学本质

Reward Hacking 通常表现为模型发现了一个极其生僻、反直觉但能骗取高分的动作序列 a_{hack} (例如在推荐系统里面我们设置奖励是用户停留时间, 但发现用户在色情、暴力的界面停留时间长, 助长了奖励)。

对于这种 a_{hack} , 其特征是:

1. **代理回报极高:** $\tilde{R}(s, a_{hack}) \gg 0$
2. **SFT 概率极低:** 由于 a_{hack} 不符合正常人类语言逻辑, 参考模型 π_{ref} 给它的概率极低, 即 $\pi_{ref}(a_{hack}|s) \rightarrow 0$ 。

KL 项的“反黑客”机制

现在, 让我们看看当模型试图通过提高 $\pi_{\theta}(a_{hack}|s)$ 来利用这个漏洞时, 总回报 R_{total} 会发生什么变化。

假设模型将 a_{hack} 的概率推高到显著水平 (例如 $\pi_{\theta}(a_{hack}|s) \approx 1$)。此时 KL 惩罚项的变化如下:

$$\text{Penalty} = -\beta \log \frac{\pi_{\theta}(a_{hack}|s)}{\pi_{ref}(a_{hack}|s)} \approx -\beta \log \frac{1}{\epsilon} \quad (\text{其中由于概率差别悬殊, 使得 } \frac{1}{\epsilon} \rightarrow \infty)$$

因此: $\text{Penalty} \rightarrow -\infty$

结论推导: 即使 $\tilde{R}(s, a_{hack})$ 非常大 (例如 100 分), 只要该动作严重脱离了 π_{ref} 的分布, KL 惩罚项 (例如 -10000 分) 就会瞬间吞噬掉所有奖励收益。

$$R_{total} = \underbrace{\tilde{R}(s, a_{hack})}_{\text{Hacked Reward (High)}} - \underbrace{\beta \log \frac{\pi_{\theta}}{\pi_{ref}}}_{\text{KL Cost (Huge)}} < 0$$

因此, 在数学上, 优化器会发现: **去触碰那些“非人类语言”的高分区域, 得不偿失。**

损失函数

$$\mathcal{L}_{total} = \mathcal{L}_{policy} + \beta \cdot \mathcal{L}_{KL}$$

对于 GRPO 来说, 因为本身也就没有 Critic 网络, 所以和 PPO 比起来就没有 $L_t^{VF}(\theta)$ 这一项, 而且 GRPO 损失函数也包含了 KL 散度, 因此也就没有再额外增加熵了。

12.6.3. 直觉理解与物理意义

物理直观理解

GRPO 的组内寻找奖励的方式, 本质思想是**输出的好不好, 不是靠一个绝对分数, 而是靠同一组里谁更好**。就是**矮子里面拔大个儿**, 也许模型输出的这一组回答都不咋样, 但还是要选择一个最好的, 即便它可能绝对值不好 (鸡头)。

动态基线的妙处

在数学原理上, $\hat{A}_i = \frac{r_i - \bar{R}}{\sigma_R}$ 利用了**蒙特卡洛采样来近似期望**。

- **降低方差**: 对于一道极难的题, 可能所有输出得分都很低。如果用绝对分数, 模型收到的全是负反馈, 效果就不好。但在 GRPO 中, 只要你比同组的其他答案稍微好一点, 你也能获得正向的梯度更新。这保证了**学习信号的稳定性**。
- **无需 Critic 的本质**: Critic 的本质是**拟合期望**。当 G 足够大时, 样本均值 $\bar{R}$ 就是奖励的无偏估计。GRPO 用**计算换空间**(多采样几次输出, 省去 Critic 显存)。

12.7. 第三部分 Measuring Zero-Shot MATH Performance

第三部分的目标是先去检测一下模型在 Zero-shot 的情况下数学能力。

Extension 12.6 什么是 zero shot 和 few shot?

- **Zero-shot**: 不给例子, 直接让模型做
- **Few-shot**: 给少量例子, 让模型照着学再做

模型在没有任何提示的情况下直接完成任务就是零样本, 比如:

Prompt: ‘把下面这句话翻译成英文: 今天天气很好’

而先给模型几个示例, 让它**学你的格式 / 风格 / 规则**, 再完成新任务, 这就是

少样本, 比如: Prompt:

‘中文: 你好英文: Hello’

‘中文: 谢谢英文: Thank you’

‘请翻译: 中文: 今天天气很好英文:’

作业要求就用 vLLM 框架实现批量推理

1. **构建 Prompt 模版**: 作业使用了 deepseek 的 ‘r1_zero’ 模版。模版强制模型在输出答案前先生成 ‘<think>’ 标签进行推理, 最后在 ‘<answer>’ 标签中输出最终结果, 格式化为了方便后续去评估 (不过模型不一定就完全按照 prompt 的要求来)。
2. **初始化 vLLM 推理引擎**: 作业使用了 Qwen2.5-MATH-1.5B, 直接在 hugging-face 上下载即可。
3. **批量生成**: 读取 ‘validation.jsonl’ 数据集 (参考 github 上疑似斯坦福的某位同学的 repo), 设置采样参数 (Temperature=1.0, stop_token=‘</answer>’), 调用 ‘llm.generate()’ 获取模型输出。
4. **答案解析与评分**: 用评分函数将模型生成的文本与标准答案 (Ground Truth) 进行比对。该函数会检查**格式是否正确**以及**答案数值是否正确**。
5. **结果序列化与分析**: 将输入问题、模型生成的完整文本、评分结果保存到磁盘, 并统计各类错误 (格式错误 vs 答案错误) 的比例。(我的结果里几乎没有几个答案正确。。)

12.7.1. 实现细节

首先给出的模板, 比如说 r1_zero_shot:

A conversation between User and Assistant. The User asks a question, and the Assistant solves it. The Assistant first thinks about the reasoning process in the mind and then provides the User with the answer. The reasoning process is enclosed within <think> </think> and answer is enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>. User: question Assistant: <think>

我们要把 question 用实际的问题去替换，在讲义里也就是用 validation.jsonl 的问题。

然后利用 vllm 框架去实现加载模型，进而批量推理。

vllm 加载模型是在 huggingface 上下载，我个人的一个加速下载的小 trick 是

1. 直接拥抱镜像站：

2. 确保 ‘pip install -U pysocks’ python 可以利用 socket5 来下载东西：‘pip install -U pysocks’

记得要配置 export ALL_PROXY=”socks5h://127.0.0.1:20170” 使得 hf 走 socks5h 协议，这样才可以利用基于 python 下的 hf 去下载东西，比如 hf download Qwen/Qwen3-4B-Base

12.8. 第四部分 SFT Helper

这一章主要关于 SFT 微调的一些实现

这一章的作业主要是实现 SFT 的一些组件。

12.8.1. 作业 1 ‘tokenize_prompt_and_output’ 要求：

构建符合 “下一个 Token 预测” 任务格式的训练数据，并确保模型只学习如何生成 “回答”，而不去学习如何生成 “提问”。

为此，我们需要将自然语言形式的 [提问, 回答] 对，转换为模型可输入的 Token ID 序列，并构造一个 掩码 (Mask)。这个掩码用于指示损失函数：在计算梯度时，忽略提问部分 (Prompt) 和填充部分 (Padding)，仅计算回答部分 (Response) 的损失。

实现思路

1. 独立分词：

- 分别对 ‘prompt_str’(问题) 和 ‘output_str’(答案) 循环调用 Tokenizer 转换成 ids。
- 获取两者各自的每个元素的长度，从而找到 “答案” 在拼接序列中的起始索引。方便后续 mask 操作。

2. 序列拼接与填充：

- 我们最终要得到的 Tensor 形状是 (batchsize, padding(prompt_ids+output_ids) 后的长度), batchsize 就是输入输出列表的长度，也就是样本数，对于每一个样本我们都要先拼接在一起，再填充 tokenizer.pad_token 到最长的样本长度；
- 填充的时候，我的做法是再新建一个 (batchsize, padding(prompt_ids+output_ids) 后的长度) 的张量，再把之前求得的拼接好的列表重新输一遍。

Keynote 12.4

这里不能够分开各自填充 `prompt_ids, output_ids` 再拼接到一起, 举个例子, 我们想要的效果是 `[input_1, input_2, input_3, output_1, output_2, pad_token, pad_token]`, 如果分开 `padding`, 再拼接, 结果就是 `[input_1, input_2, input_3, pad_token, output_1, output_2, pad_token]`, 顺序出了问题。

3. 构建 `mask`:

- 创建一个与 ‘full_sequence’ 等长的 bool 张量 (或者 0,1 也行)。
- **Prompt 部分**: 设为 ‘False’。
- **Output 部分**: 设为 ‘True’。
- **Padding 部分**: 设为 ‘False’。

4. 构建自回归训练数据:

- **input_ids**: 取序列的 第 0 个到倒数第 2 个 Token (切去最后一个)。
- **labels**: 取序列的 第 1 个到最后一个 Token (切去第一个)。
- **response_mask**: 从 `mask` 里进行相应的切片, 与 ‘labels’ 对齐。

具体示例

假设 Tokenizer 词表映射为: “A”: 1, “B”: 2, “C”: 3, “D”: 4。

- **Prompt**: “A B” -> ids: ‘[1, 2]’
- **Output**: “C D” -> ids: ‘[3, 4]’

数据流转如下:

1. **完整序列**: ‘[1, 2, 3, 4]’ (对应 “A B C D”)
2. **原始掩码**: ‘[0, 0, 1, 1]’ (0 代表 Prompt “AB”, 1 代表 Output “CD”)
3. **切片处理**:

- **input_ids**: ‘seq[:-1]’ -> ‘[1, 2, 3]’ (输入 A, B, C)
- **labels**: ‘seq[1:]’ -> ‘[2, 3, 4]’ (预测目标 B, C, D)
- **response_mask**: ‘mask[1:]’ -> ‘[0, 1, 1]’
 - 第一个 ‘0’ 对应 label ‘2(B)’: 这是 Prompt 的一部分, 我们不希望计算 loss, 不强迫模型学习 Prompt。
 - 第一个 ‘1’ 对应 label ‘3(C)’: 这是 Output 的第一个词, 要计算 loss。
 - 第二个 ‘1’ 对应 label ‘4(D)’: 这是 Output 的第二个词, 要计算 loss。

12.8.2. 作业 2‘compute_entropy’ 要求:

计算 LLM 在生成每个 Token 时的**预测熵；逐 Token 熵 (Per-token Entropy)**是对模型在特定位置生成的概率分布**不确定性的定量度量**(在信息论里熵本身就代表了不确定性)；熵值越高表示模型对下一个 Token 的选择越困惑，熵值越低则表示模型越趋向于输出某些特定 Token(即预测越自信)。

实现思路

输入是**Logits**，即经过深度学习模型最后一个线性层输出的**未经归一化的原始分值向量**。

1. **Logits 归一化**: 输入 ‘logits’ 的维度为 (B,L,V)。首先需将其通过 Softmax 函数转换为概率分布 $p(x)$, 公式为:
$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$
2. **对数概率计算**: 计算 $\log p(x)$ 。为避免数值溢出，不直接对 Softmax 结果取对数，而是使用 ‘F.log_softmax’ 或通过公式 $\log p_i = z_i - \text{logsumexp}(z)$ 直接从 Logits 计算。(具体原理细节可详见第一章笔记)
3. **应用熵公式**: 根据定义 $H(p) = -\sum_{x \in X} p(x) \log p(x)$, 将每个 Token 位置对应的词表维度 (Vocab) 进行加权求和。
4. **维度压缩**: 在最后一个维度 (‘vocab_size’) 上进行求和，最终输出维度为 (B,L) 的张量，其中每个元素代表该位置的预测不确定性。

12.8.3. 作业 3‘tokenize_prompt_and_output’ 要求:

给定一个因果语言模型 (Causal LM) 和一段文本序列，计算模型生成该序列中**特定 Token(即标签 Labels)**的**条件对数概率**，并可以选择性地返回我们在上一步计算的**熵 (Entropy)**。

实现思路

1. **获取原始 Logits**: 将 ‘input_ids’ 输入模型，得到 ‘logits’。其维度为 (B,L,V)，其中 B 是 Batch Size, L 是序列长度, V 是词表大小。
2. **对数归一化**: 对 ‘logits’ 使用 ‘log_softmax’ 操作，将原始分值转换为对数概率 $\log p$ 。这一步必须在词表维度 (‘dim=-1’) 上进行。
3. **对齐标签**: 在因果语言模型中，位置 t 的 Logits 是用来预测位置 t+1 的 Token 的。因此，为了获取 ‘labels’ 中每个位置的对数概率 (labels 指明了哪些位置的 token 是和训练对应的): - 需要将 ‘log_probs’ 的前 L-1 个位置与 ‘labels’ 的后 L-1 个位置对应。 - 或者根据具体框架实现，确保取出的概率值 $p(y_t)$ 是基于序列 $x < t$ 计算得出的。
4. **提取目标 Token 的概率**: 使用 ‘torch.gather’ 函数，根据 ‘labels’ 提供的索引，从 (B,L,V) 维度的对数概率张量中提取出对应的标量值。

5. **计算熵 (optional)**: 如果 `'return_token_entropy'` 为 `'True'`, 调用之前实现的 `'compute_entropy'` 函数, 并将其与对数概率一起存入字典。

具体示例

假设输入 `'input_ids'` 为 `'[I, love, AI]'`, 对应的 Token ID 为 `'[10, 25, 40]'`:

1. 模型在位置 `'I'` 输出的分布中, 我们提取 Token `'love'` (ID 25) 的对数概率。
2. 模型在位置 `'love'` 输出的分布中, 我们提取 Token `'AI'` (ID 40) 的对数概率。
3. 最终返回的结果将是 `'[log_p(love|I), log_p(AI|I love)]'`。

12.8.4. 作业 4 `'mask_normalize'` 要求:

根据一个布尔掩码对输入张量的指定元素进行求和, 然后将该和值除以一个常数进行归一化。非常 easy。

12.8.5. 作业 5 `'get_response_log_probs'` 要求:

对一个**微批次 (micro-batch)**数据执行一次完整的**前向计算 (计算损失)**和**反向传播 (计算梯度)**。

1. 计算掩码损失:

- 输入 `'policy_log_probs'` 是模型对于每个 token 给出的对数概率, 其形状为 `'(batch_size, sequence_length)'`。
- 输入 `'response_mask'` 是一个与 `'policy_log_probs'` 形状相同的张量, 其中 **1** 代表这是模型需要学习和生成的“回答”部分, **0** 代表这是输入的“提示”或填充部分。
- 我们的目标是让模型只学习“回答”部分。因此, 需要将 `'policy_log_probs'` 与 `'response_mask'` 进行**逐元素相乘**, 这样“提示”部分的对数概率就全部变为 0, 不会对总损失产生贡献。

2. 标准化损失:

- 将上一步得到的总和除以 `'normalize_constant'`。

3. 梯度累积缩放:

- 因为我们最终要累积 `'gradient_accumulation_steps'` 次梯度, 为了使最终累积的梯度在数值上等价于单个大批次的梯度, 每次计算出的微批次损失需要**除以** `'gradient_accumulation_steps'`。

4. 反向传播:

- 对缩放后的损失 `'scaled_loss'` 调用 `'backward()'` 方法。PyTorch 会自动计算出模型参数关于这个 `'scaled_loss'` 的梯度, 并将其存储在各个参数的 `'grad'` 属性中。由于我们不会在每次微批次后清空梯度, 这个新计算的梯度会自动累加到已有的梯度上。

12.8.6. 梯度累积

梯度累积 (Gradient Accumulation) 是一种优化显存的技术, 通过将一个大的全局批次拆分为多个微批次分次执行前向与反向传播, 并将计算得到的梯度在内存中进行线性叠加, 直到达到预设的累积步数后才执行一次参数更新, 从而在数学上等效于在大批量数据下进行训练。本质就是一种以时间换空间的做法。

技术原理

梯度累积利用了深度学习框架中梯度默认累加的特性。其执行流程如下:

1. **批次拆分**: 设定目标全局批次大小和物理微批次大小。计算累积步数: $\text{Accumulation Steps} = \frac{\text{Global Batch Size}}{\text{Micro-batch Size}}$
2. **标准化损失计算**: 在每个微批次的前向传播计算出 Loss 后, 必须将其除以累积步数。 $\text{Scaled Loss} = \frac{\text{Loss}}{\text{Accumulation Steps}}$
3. **反向传播与累积**: 执行反向传播。此时框架计算当前微批次的梯度, 并将其直接加到模型参数的 '.grad' 属性上, 而不是替换原有的梯度。在此过程中, **不执行优化器的更新和清零操作**。
4. **参数更新**: 重复 2 和 3, 直到执行完指定次数的微批次。此时, 模型参数中存储的梯度即为整个全局批次的梯度总和。调用优化器更新模型权重。
5. **梯度清零**: 权重更新完成后, 清空所有参数的梯度, 准备开始下一个全局批次的循环。

具体示例

假设显存仅能支持 Batch Size = 4, 但为了保证模型收敛稳定性, 需要使用 Batch Size = 32 进行训练。

参数设定:

- 物理限制 (Micro-batch): 4
- 目标批次 (Global Batch): 32
- 累积步数: $32/4=8$

数据流转:

1. **循环执行**:
 - 加载 4 条数据。
 - 前向传播计算 Loss。
 - $\text{Loss} = \text{Loss} / 8$ 。
 - 反向传播, 计算梯度 g_i , 累加到显存: $G_{total} = G_{total} + g_i$
 - **注意**: 此时权重 W 保持不变。
2. **更新步骤**:

- 加载最后 4 条数据。
- 前向传播, Loss / 8, 反向传播。
- 此时 G_{total} 包含了 32 条数据的梯度信息。
- 执行 Optimizer Update: $W_{new} = W_{old} - \eta \cdot G_{total}$
- 执行 Optimizer Zero Grad: $G_{total} = 0$

12.9. 后训练概要

12.9.1. 后训练与预训练的区别

预训练 (Pre-training)指模型构建初期在海量无标注数据上训练, 以学习通用特征和语言知识。预训练通常使用上百亿甚至数万亿规模的 token, 通过**自监督任务 (如下一个词预测)**让模型掌握语言的基本规律。这一阶段产生的基础模型具备广泛的语言能力, 但不一定懂得遵循人类指令。

后训练 (Post-training)则发生在预训练之后、模型部署之前, 用特定任务或目标数据对模型进行额外训练。后训练包含**微调 (Fine-tuning)**和**对齐 (Alignment)**过程, 通常涉及**有监督微调**和**基于人类反馈的强化学习 (RLHF)**等方法。与预训练侧重通用性不同, 后训练聚焦在定制模型行为, 让模型适应特定应用场景或对齐人类偏好。简单来说, 预训练教会模型“能回答”, 而后训练教会模型“如何更好地回答”。两者相比:

- **数据:** 预训练用海量通用语料, 后训练用更窄的、高质量的指令或对话数据。
- **目标:** 预训练追求广泛知识获取, 后训练追求在特定任务上优化模型性能。
- **阶段:** 预训练耗费最大算力资源, 后训练相对开销小但关键, 往往决定模型最终实用效果。

12.9.2. 后训练的主要目标

后训练阶段的核心目标是**将通用大模型对齐到人类期望的行为**, 主要体现在以下几方面:

- **对齐人类偏好:** 通过后训练, 让模型不仅会回答问题, 还能回答得**有用、满足特定格式或符合人类的某些其他偏好**。例如, 让模型回答的内容满足 latex 格式。
- **增强特定能力:** **针对特定任务或领域提升模型性能**。例如, 通过指令微调让模型学会遵循复杂指令、多轮对话逻辑; 或者通过专项数据 (如代码、数学推理数据) 微调来增强模型的编程、推理等专项能力。后训练可以让模型在特定领域的回答更准确、更专业。
- **提高交互体验:** 让模型回答更符合用户预期的风格和长度, 减少无关或冗长内容。比如通过偏好对齐数据, 让模型的回答更加简洁明了或风趣友好, 提升人机交互体验。
- **安全性与可靠性:** 利用后训练减少**模型幻觉**和有害输出。通过奖励惩罚机制, 引导模型避免编造事实或使用不当言辞, 使输出更加可信且安全。

总之, 后训练的目的在于**在不改变模型底层知识的前提下, 塑造模型的输出行为**, 让其更贴近人类期望的回答方式和质量要求。

12.9.3. 主流的后训练方法及工程实现流程

后训练阶段包含多个步骤和方法, 每一步都有不同的工程实现要点。典型的后训练流水线包括**有监督指令微调 (SFT)**、**奖励模型训练 (RM)**、**基于人类反馈的强**

化学习 (RLHF) 三步。此外,也有一些新的替代方案和改进方法 (如 DPO、RRHF、SLiC-HF、RLAIF 等)。

有监督指令微调 (SFT)

有监督微调 (Supervised Fine-Tuning) 也称“指令微调”。在这一步,我们使用人工标注的 **[指令, 回答]** 示例数据集,对预训练模型进行继续训练,以培养模型的基础对话和指令遵循能力。SFT 的**训练目标仍是让模型预测下一个 token**,但训练数据是成对的指令和期望回答。通过模仿高质量人类演示,模型学会遵循指令进行响应。

SFT 所需的数据包含一个用户指令和对应的高质量回答。例如:展示了一条中文指令微调数据:

```
"input": "写一首关于猪的作文",  
"output": "从前有一只猪很懒,睡到中午才起来..."
```

之后我们要选择一个预训练好的基座模型作为初始模型,比如 Qwen、Deepseek 等,由于完整微调大模型显存开销巨大,工程上常采用 **LoRA** 等**参数高效微调 (PEFT)** 技术。LoRA 通过在模型某些权重上添加低秩矩阵,只训练这些小规模参数,从而显著降低显存和计算需求。

常用工具链有:

- **Hugging Face Transformers**: 提供模型加载、Tokenizer 和 Trainer 等便捷工具。可结合 Datasets 读取和处理指令数据集。
- **PEFT 库**: 与 Transformers 集成,用于插入 LoRA 等适配器,封装了训练这些新增参数的逻辑。
- **DeepSpeed/FSDP**: 在全参数微调大模型时,可借助 DeepSpeed ZeRO 或 PyTorch FSDP 进行模型并行和内存优化,实现多 GPU 协同训练。DeepSpeed 也支持混合精度、梯度检查点等加速和省显存技巧。

通过 SFT 阶段,模型将具备基本的指令遵循和对话能力,为后续的奖励对齐和强化学习奠定基础。

奖励模型训练 (RM)

奖励模型是一个**评分模型**,用于评价模型输出对人类偏好的符合程度。它接受“提示 + 模型回答”的输入,输出一个得分,分数越高表示回答越符合人类偏好。训练一个准确的 RM 是 RLHF 的关键步骤,因为**后续强化学习将以 RM 提供的奖励信号来优化策略模型**。

训练 RM 需要人工偏好数据,通常是**同一提示下多种回答的比较/排序**。典型做法是:

1. 使用上一步的 SFT 模型,对一批提示生成若干不同回答 (例如 4-9 个)。
2. 根据质量对这些回答进行偏好排序 (从最佳到最差)。

3. 从排序结果构造**成对偏好比较**的数据:(Prompt, 回答 A, 回答 B, 标签), 其中标签指示 A 是否优于 B。通常把人工认为更好的回答作为‘**chosen**’, 较差的作为‘**rejected**’, 形成二元比较对。

例如一个偏好数据可以格式化为:

```
"prompt": "给小学生解释什么是太阳能。",  
"chosen": "太阳能是来自太阳的能量...(通俗易懂正确的解释)",  
"rejected": "太阳能是一种魔法, 可以让灯泡亮起来。(错误或不当的回答)"
```

开源的偏好数据集通常以这样的‘**chosen**’/‘**rejected**’字段形式提供。

奖励模型通常基于预训练模型构建, 以确保理解复杂语言输入的能力匹配。但它在 LM 顶部增加一个回归头用于输出一个标量分数。具体实现上, 可以复制 SFT 微调后的模型权重作为初始权重, 然后修改最后一层为 1 维输出。这样 RM 的输入是完整的 (prompt+answer) 序列, 输出一个 logits 值作为评分。

使用前述偏好比较数据, 通过**监督学习**训练 RM。例如采用**对比损失**: 对于每对 (chosen, rejected), 令模型输出的分数为 S_c 和 S_r 。我们希望 $S_c > S_r$, 且差距越大越好, 这就代表我们想要的结果和干扰的噪声结果区分度更大, 这相当于把偏好比较当作**二分类任务**: 模型判断 “chosen 是否优于 rejected”。训练过程中可将同一 Prompt 下的每个正负回答对视作一个训练样本, 计算上述损失并反向传播调优 RM 参数。

常用损失是 Logistic 回归损失或对数 sigmoid 损失:

$$L = \log(S_c S_r)$$

其中 σ 是 sigmoid 函数。

训练得到的 RM 需要在之后的强化学习阶段保持**权重冻结**, 作为固定的奖励评估器使用。

如果 RM 不准确, 会直接导致策略模型学习到错误的优化方向。

基于人类反馈的强化学习 (RLHF)

整体流程: 在拥有 SFT 微调模型 (作为初始策略) 和训练好的奖励模型之后, 即可进入 RLHF 阶段, 即用**强化学习进一步微调模型参数**, 使其倾向于输出高奖励的响应。常用算法是**PPO**, 一种策略梯度方法。RLHF 通过试错式训练进一步提升模型回答质量和对齐程度, 被广泛证明能显著提高模型在人类偏好、内容安全等维度的表现。

RLHF 训练引入了几个重要的模型和概念:

- **策略模型**: 即我们要优化的语言模型。在开始 RL 时通常直接使用 SFT 后的模型参数作为初始策略 π_θ 。
- **参考模型**: 一份策略模型初始参数的冻结副本 π_{ref} 。参考模型用于计算新策略与原策略之间的差异 (**KL 散度**) 以施加正则, 防止策略偏离初始行为过多。

- **奖励模型**: 前一步训练得到的 RM, 用来对策略模型生成的回复给出奖励分数 R , 指引策略优化方向。
- **价值函数**: Critic 网络, 近似估计每个状态 (或每个 token 位置) 的预期累计奖励。它的作用是用于**计算优势函数** $A=R-V$, **减少策略梯度的方差**。

在工程实现中, 我们会:

- 复制一份 SFT 模型权重作为‘**ref-model**’冻结。
- 在 SFT 模型上添加 value head 得到‘**ppo-model**’, 此时‘**ppo-model**’有两个头: 原来的语言模型头用于生成文本, 新的价值头用于估值。
- 训练循环: PPO 训练通常以下面的方式迭代进行:
 1. **生成数据**: 从训练的提示集合中抽取一批 prompts, 用当前策略模型 (‘**ppo-model**’) 对每个 prompt 生成回复 (通常采用一定随机性如**温度采样**, 以促进策略探索不同回答)。生成长度可限制在一定范围以控制计算成本。
 2. **计算奖励**: 对每个 prompt 的生成回复, 使用**冻结的 RM**打分, 得到奖励分数 R 。此外, 计算新回复相对于参考模型的**KL 惩罚**: 即统计‘**ppo-model**’在生成回复上的概率分布与‘**ref-model**’分布的 KL 散度。通过 RM 得分和 KL 值, 可构造一个**正则化后的奖励**: $R' = R - \beta \cdot \text{KL}$, 其中 β 是惩罚系数。这表示模型在得到高奖励的同时会因偏离参考而被扣分, 从而平衡探索和保守。
 3. **优势估计**: 使用价值函数, 计算每个生成序列的值估计 V , 并基于最终的 R' 计算优势 $A = R' - V$ 。对于序列中每个 token 位置, 也可以计算每步的优势, 但通常简化为把最终总优势赋给序列中的各 token (使用**GAE**等方法平滑计算)。
 4. **PPO 更新**: 将生成的 prompt 和模型输出 (以及计算出的优势 A 和参考概率分布) 作为一次**交互轨迹**, 基于 PPO 算法更新策略模型参数。PPO 的损失包括: 策略损失 (提高高优势动作的概率, 降低低优势动作概率, 且用剪切机制限制更新幅度)、值函数损失 (使价值头预测更贴近实际 R') 以及策略与参考的 KL 正则项。通过反向传播调整 π_θ 的参数。
 5. **重复迭代**: 更新后, 用更新后的策略继续采样新数据, 不断循环。

RLHF 也存在一些典型问题:

- **Reward Hacking**: 模型可能学到投机策略骗过了我们预期的 RM, 即出现了**Reward Hacking**。这会导致我们模型跑偏。比如我们短视频的推荐算法奖励是用户停留时长, 即看的时间越长, 越会给用户推荐此类视频。但一不小心出现了色情暴力的视频, 用户停留时间长满足了奖励, 但这显然不是我们想要的结果。
- **不稳定性**: 相比监督学习, RL 训练的收敛更不稳定、调参难度高。不同随机种子、奖励权重都会影响结果。需要较多试错和经验去达到理想效果。

其他后训练方法及其工程特点

除上述经典的 SFT-RM-PPO 流程外，业界和学术界也提出了一些**替代性的后训练方法**，旨在提高训练效率、稳定性或减少对人类标注的依赖。

- **直接偏好优化 (DPO)**: Direct Preference Optimization 是一种**无需显式奖励模型 RW 和在线 RL**的偏好对齐方法。DPO 使用离线收集的**偏好比较数据**(与 RM 训练相同的数据)，但直接通过跟偏好相关的损失函数来微调模型，使其对人类偏好更高的回答给出更高概率。

具体而言，DPO 保留一个参考模型 (SFT 模型参数冻结) 作为基准，用损失函数鼓励模型对于‘**chosen**’回答的生成概率高于参考模型，对‘**rejected**’回答的概率低于参考模型。这样模型在不经 PPO 的情况下，就隐式地优化了偏好差值，同时通过参考模型起到 KL 正则作用。

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

- **RRHF**: Rank Responses to Align Language Models with Human Feedback, 是另一种离线偏好对齐方法。RRHF 利用**多个候选回复的排序信息**训练模型：对于同一 prompt，模型对不同响应计算长度归一化的序列概率，对比这些概率的大小与人类偏好顺序，使模型调整自身概率分布来匹配人类偏好。

- 奖励模型 (RM) 评分：‘Score(R1)=0.9’，‘Score(R2)=0.3’，‘Score(R3)=0.7’

- 处理流程：

1. RM 评分排序为：R1 > R3 > R2。

2. 计算损失：

- 比较 (R1, R2): 模型为 R1 赋予的概率应高于 R2, 否则产生损失。
- 比较 (R1, R3): 模型为 R1 赋予的概率应高于 R3, 否则产生损失。
- 比较 (R3, R2): 模型为 R3 赋予的概率应高于 R2, 否则产生损失。

3. 将所有这些成对的损失加总，形成最终的 batch loss。

- **RLAIF(Reinforcement Learning from AI Feedback)**: 这是用**AI 替代人类反馈**的范式。RLAIF 流程与 RLHF 一样，唯一区别是把人类偏好标签换成由强大的 AI 教师模型来自动生成。

12.10. 系统梳理 GRPO 的整个训练过程

12.10.1. 采样 (sample)

首先，对每个训练样本（例如每个 **prompt** 问题）让当前的旧策略模型生成多个不同的回答（**response**）。通常我们为每个 **prompt** 采样固定数量的输出，例如 8 个，以形成一个 **候选回答组 (group)**。假设原始数据集中有 100 条 **prompt**，那么采样后得到的回答矩阵形状就是 (100, 8)，即每个问题对应 8 个模型回答。这些回答都是依据 **旧策略**（即当前模型参数）独立采样得到的。

Keynote 12.5

在这里，“旧策略”指的是我们用于采样的模型策略参数（在训练过程中通常是前一轮优化后的策略，用作参考策略），而“新策略”指的是将要更新的模型参数。每轮采样都是基于旧策略来生成数据。

典型地，**组大小 G** 可以取 4 到 8 之间；研究表明在大型 Transformer 模型上，采用 4~8 个候选就足以取得良好的效果，进一步增大组的规模会继续降低梯度方差但回报递减。

12.10.2. 评分

对每个 **prompt** 获得的一组 8 个回答，我们需要评估它们的质量，从而得到奖励值 (reward)。在 GRPO 框架中，可以使用 **外部的评分函数** 或 **奖励模型** 对回答进行评价。评分标准取决于具体任务，例如在 LLM 微调中可以依据回答的 **正确性** 和 **格式规范** 给出分数。如果有 **标准答案 (ground truth)**，我们可以自动比较回答与标准答案的匹配程度并衡量正确性；对于格式要求明确的任务，也可以设计规则检查回答格式是否合规。这些评估得到的 **原始奖励** 记作 **raw_rewards**，其形状与采样结果一致 (如 100×8)，即每个问题的 8 个回答各有一个分数。

12.10.3. 组内归一化 (计算优势 Advantage)

有了每个回答的 raw reward 之后，GRPO 的关键一步是在 **每个 prompt 的回答组内对奖励进行归一化**，以计算出 **优势函数 (advantage)** 值。具体做法是：**对同一 prompt 下的多个回答，其奖励减去该组奖励的平均值，再除以该组奖励的标准差**（通常会加上一个很小的常数 δ 防止除零）。用公式表示，每组中的第 i 个回答的优势为：

$$A_i = \frac{r_i - \mu_{\text{group}}}{\sigma_{\text{group}} + \delta}$$

其中 r_i 是第 i 个回答的原始得分， μ_{group} 是该 **prompt** 下 8 个回答得分的平均值， σ_{group} 是它们的标准差。这样归一化后的优势值 A_i 反映了每个回答 **相对于同组其他回答的好坏程度**：高于 0 表示该回答比组内平均水平好，低于 0 表示比平均差。

采用 **组内优势归一化** 有两个好处：

- **消除分数尺度的影响**：归一化使我们关注的是 **相对排名** 而非 **绝对分值**，不管原始打分函数的取值范围如何，归一化后每组内奖励的均值为 0，方差为 1。

这样能够确保不同 prompt 之间的奖励分布具有可比性，也不受评分偏置的影响。模型只需判断“一组候选中哪些回答更优”即可。

- **降低梯度方差，稳定训练**：这实际上提供了一种无需值函数基线的优势估计，相当于对策略梯度做了**减去 baseline**的处理，从而减少了策略梯度的方差。传统 PPO 依赖训练一个价值网络 $V(s)$ 来估计 baseline，而 GRPO 通过组内比较**省去了训练价值函数的繁琐**。这不仅简化了算法（降低内存和算力开销），也提高了采样效率和稳定性。直观来说，**每次策略更新都会让模型更倾向于产生组内相对更好的回答，降低生成较差回答的概率**。

12.10.4. 模型训练准备（数据序列化与概率计算）

经过以上步骤，我们已经得到了用于训练的数据：每个 prompt 的问题文本、模型生成的一个回答（将对 8 个回答分别进行训练）、以及对应的归一化优势值 A 。接下来，需要将文本和优势结合起来用于模型参数的更新。以下是这一阶段的具体步骤：

文本序列化与 Tokenization

将 prompt 和对应的模型回答拼接成完整输入序列。例如，可以采用格式：“<|prompt|> 问题内容... <|response|> 回答内容...”。然后对这个序列进行**分词**，转换为模型可处理的 token ID 表示。为了构造训练样本，我们通常将**prompt+response 作为一个整体序列**，并利用因果语言模型的方式来设置输入和标签：

- **input_ids**：取上述序列的所有 token，但去掉最后一个 token（序列从开头到倒数第二个 token）。这些将作为模型的输入序列，使模型在看到该 token 序列的前缀下去预测下一个 token。
- **labels**：取同一序列的所有 token，但去掉第一个 token（序列从第二个 token 到最后一个 token）。这样 labels[i] 实际上就是原始序列中 input_ids[i] 之后应该预测的下一个 token。模型在训练时会尝试用第 i 个输入预测出第 $i+1$ 个 token。

这个“一位移”的设置确保模型的**预测目标**对齐为“给定前面的文本，预测下一个词”，符合因果语言模型训练的要求。例如，如果序列是“A B C”，那么 input_ids = [A, B]，而 labels = [B, C]，模型接受“A”预测出 B，接受“A B”预测出 C。

此外，我们还需要一个**响应的掩码 (response mask)**。因为 prompt 部分是给定的上下文，我们不希望在计算损失时将 prompt 部分也算进去。通过构造一个 mask，将 prompt 部分给 mask 掉而**只计算回答部分的预测损失**。这样可以确保优化的梯度仅来自模型生成的内容部分，而不会影响对提示的处理。这个 mask 会与 labels 对齐，mask 掉序列中属于 prompt 提示的部分，只保留 response 部分参与策略梯度计算。

计算对数概率

有了 `input_ids` 和 `labels` 后，我们通过模型的前向传播计算**对数概率**，为后续的策略梯度公式做准备。具体步骤如下：

1. **获取原始 Logits**：将 `input_ids` 喂入当前策略模型，得到每个位置上的 logits 输出。Logits 是模型未归一化的原始得分，一般维度是 (B, L, V) ，其中 B 是批量大小， L 是序列长度， V 是词表大小。每个 $\text{logit } l_{t,v}$ 表示模型在位置 t 上输出词汇表中第 v 个词的原始分值。
2. **对数归一化 (Log Softmax)**：对 logits 沿着词表维度应用 `log_softmax` 函数，将其转换为对数概率。也就是先通过 `softmax` 将每个位置的 logit 变成概率分布，再取对数得到 $\log p$ 。经过这一步，我们得到张量 `log_probs`，其形状仍是 (B, L, V) ，但每个位置 t 上的向量变成了该位置输出每个词的对数概率 $\log p_{t,v}$ 。
3. **标签对齐**：由于自回归语言模型在位置 t 的输出是用于预测序列中第 $t+1$ 个 token（如前述的 `labels` 设置），我们需要将计算得到的对数概率与正确的目标标签对应起来。具体地，**对于序列长度为 L 的输入，位置 0 的 logits 用于预测位置 1 的标签，位置 1 的 logits 用于预测位置 2 的标签，...，位置 $L-2$ 的 logits 用于预测位置 $L-1$ 的标签**。因此我们通常会丢弃序列最后一位的 logits（没有下一个 token 可预测），或者在代码实现上，通过对 `labels` 移位来对齐。这一步确保 `log_probs` 中的第 t 行对应 `labels` 中的第 t 项的对数概率。
4. **提取目标 token 的对数概率**：现在，对每个序列位置，我们从 `log_probs` 中取出模型对正确标签的对数概率值。这可以使用诸如 `torch.gather` 的操作：根据 `labels` 提供的索引，从每个位置的概率分布向量中选出对应标签 token 的概率。最终得到一个形状为 $(B, L-1)$ 的矩阵，每个元素是模型在该位置输出正确下一个词的 log 概率 $\log \pi_{\theta}(y_t | x_{<t})$ 。

经过上述步骤，我们就拿到了模型对每个 prompt 对应回答的概率信息。其中关键是：**新策略 π_{θ} 输出当前回答序列的概率**，以及**旧策略 π_{old} 输出同一回答的概率**。由于在实际实现中，我们通常会保留旧策略模型以便同时计算 $\pi_{\text{old}}(o_i | q)$ ，从而后续构造比率。对于训练第一轮来说，旧策略和新策略参数是相同的（模型还未更新），所以两者概率相同。

策略损失计算（PPO 剪辑目标）

最后，我们使用前面得到的概率值和优势 A 来计算策略梯度的**损失函数**，并通过反向传播更新模型参数。GRPO 采用了与 PPO 类似的**剪辑策略梯度目标**。具体来说，对于每个样本（prompt）下的第 i 个回答，我们计算**概率比率 ρ_i** 和剪辑后的目标：

- **概率比率**：定义 $\rho_i = \frac{\pi_{\theta}(o_i | q)}{\pi_{\text{old}}(o_i | q)}$ ，也就是新策略模型在 prompt q 上生成回答 o_i 的概率与旧策略模型生成该回答概率的比值。这个比率衡量新策略相对于旧策略在该回答上的概率放大或缩小的程度。如果 $\rho_i > 1$ ，说明新模型比旧模型更倾向于产生这个回答；反之若 $\rho_i < 1$ 则倾向降低其概率。

- **剪辑策略目标**：PPO 引入剪辑的思想来避免单次更新幅度过大。我们计算一个未剪辑的目标项和一个剪辑后的目标项： $L_{PG,i} = \min(\rho_i A^i, \text{clip}(\rho_i, 1, 1+\epsilon) A^i)$.
- **损失与梯度**：我们希望最大化上述目标期望，也就是让策略更偏向优势为正在的动作，远离优势为负的动作。通常将上述目标取负号变为损失函数供优化器最小化。也即，策略梯度的损失定义为： $\mathcal{L}_{\text{policy}} = -\mathbb{E}_i [L_{PG,i}]$

其中对每个样本的各个候选 i 取平均（或者求和）再取负号。在实现中，这相当于：
如果优势 A_i 为正，模型将受到负梯度推动去增大对应回答的概率（因为此时 ρA 取负后是负的梯度，梯度下降会提高该概率）；如果优势为负，模型则被推动降低该回答概率。这与策略梯度 (REINFORCE) 的直观作用是一致的，只不过借助了旧策略来衡量概率比率并进行了剪裁调制。

值得注意的是，在训练开始的第一次迭代，我们通常设置旧策略 π_{old} 与新策略 π_{θ} 权重相同（因为还没更新模型）。此时对所有 i 都有 $\rho_i \approx 1$ ，剪辑也不起作用，于是损失近似为 $-A_i$ 的形式。也就是说，第一步基本就是根据优势的符号来调整策略：正优势让策略概率增大，负优势让其减小。随着若干步更新后，我们会定期更新旧策略（例如每个 epoch 或一定步数后将当前策略保存为新的参考策略），继续上述过程，直到收敛。

12.10.5. 总结

总结来说，GRPO 训练过程包括：对每个 prompt 用旧模型采样多条回答，计算每条回答的奖励分数，然后组内归一化得到优势，再将 (prompt + 回答) 序列输入模型计算对数概率，最后依据剪辑的策略梯度公式来调整模型参数。通过组相对优势的引入，GRPO 避免了训练价值函数，以更低的开销实现了与 PPO 类似的效果。

12.11. DPO 深入原理推导及训练实现

12.11.1. DPO 是什么，为什么我们需要它？

直接偏好优化 (Direct Preference Optimization, DPO) 是一种用于**大语言模型对齐 (Alignment)**的技术，旨在让模型输出更符合人类偏好。与 RLHF 的方法不同，DPO **不需要单独训练奖励模型**，而是直接利用人类偏好数据对模型进行微调。

Keynote 12.6

即 **DPO** 的数据形式是 (x, y^+, y^-) 这种，明确告诉你 y^+ 好， y^- 不好，调 **policy** 目标就是为了让它更偏向 y^+ 而远离 y^- 。说白了，这就已经不是强化了，而是监督学习。所以，容易理解，这个 **DPO** 适合的场景就是那种二选一的，比如两个商品去推断哪个用户更喜欢。

RLHF 过程复杂且不稳定，需要多个阶段（**监督微调、奖励建模和强化学习**）才能完成模型优化。具体来说，RLHF 方法存在以下挑战：

- **训练复杂度高**：RLHF 包含训练奖励模型并在策略优化中反复采样模型输出进行强化学习。训练过程需要精心设计的奖励函数、策略梯度算法（如 **PPO**），实现和调参难度大。
- **不稳定性**：强化学习阶段往往不够稳定，相同的数据和超参数可能出现截然不同的结果。模型在 RL 过程中可能发生 **Reward Hacking** 或策略坍塌等问题，需要引入 KL 惩罚项保持模型行为不要偏离初始模型太远。
- **计算成本高**：由于每一步优化都依赖于奖励模型的反馈，RLHF 训练往往需要反复生成样本评估，涉及四个模型协同（策略、参考、奖励、价值模型），使单步训练耗时且资源占用大。

而 DPO 计算开销更低、实现更简单，但在对齐效果上却不输 RLHF。DPO 从理论上将 RLHF 中的奖励优化问题转化为了一个**简单的偏好分类问题**，**省略了显式的奖励模型和强化学习环节**。

12.11.2. 简要回顾 SFT、Reward Model、RLHF 经典 post-training 流程

- **监督微调 (SFT)**：指在预训练大模型的基础上，使用**高质量有标注的数据**对模型进行**有监督训练**，使其更好地完成下游任务。例如 ChatGPT 的训练第一步，就是用大量人工编写的**指令-回答示例**对模型进行微调，得到一个顺从指令的 SFT 模型。这个 SFT 模型在回答形式、语气上更接近人类期望，但仍可能低质量的输出。
- **奖励模型 (Reward Model)**：为了进一步让模型输出符合人类偏好，研究者会让模型针对各种提示生成多个候选回答，然后请人类标注哪一个回答更好。这些人工比较结果形成了**偏好数据对**。有了这些偏好对，传统 RLHF 方法会训练一个**奖励模型 (RM)**，其实质是一个**二分类模型**，它输入模型输出文本，给高质量回应打高分，劣质回应打低分。

- **人类反馈强化学习 (RLHF)**：在这一步，使用上面的奖励模型来指导策略模型（通常是 SFT 模型的**副本**，即**Actor 模型**）进行强化学习微调。典型方法是**近端策略优化 (PPO)**：让 Actor 生成回答，根据奖励模型打分作为即时奖励，同时与一个**冻结的参考模型 (Reference Model)**对比计算 KL 惩罚，最后用策略梯度提升 Actor 对高分回答的概率、降低低分回答的概率。

SFT -> 偏好收集得到奖励函数 -> RLHF (PPO) 这是 post-training 的标准传统 pipeline。DPO 所做的改变就是它试图**绕过奖励模型和复杂的 RL 寻优，直接基于偏好数据对来优化模型参数**，实现与 RLHF 相同的目标。

12.11.3. 数学原理详细推导

核心定义与符号体系

定义以下数学符号：

- x ：输入，从数据集分布 $\mathcal{D}$ 中采样。
- y ：输出。
- y^+, y^- ：成对的偏好数据，其中 y^+ 是被人类标注者偏好的回答， y^- 是被拒绝的回答。
- $\pi_{\text{ref}}(y|x)$ ：参考策略，通常是 SFT 后的模型。我们希望优化后模型不要偏离它太远。
- $\pi_{\theta}(y|x)$ ：我们正在训练的策略网络，参数为 θ 。
- $r^*(x, y)$ ：潜在的真实奖励函数，但在 DPO 中我们不需要显式建模它。
- β ：KL 散度惩罚系数，控制策略偏离参考策略的程度。
- $\sigma(\cdot)$ ：Sigmoid 函数， $\sigma(z) = \frac{1}{1+e^{-z}}$ 。

数学原理推导

DPO 利用了**变分法 (Calculus of Variations)** 中的思想。

第一阶段：重写 RLHF 的目标函数

在标准的 RLHF（如 PPO 阶段）中，我们的目标是最大化期望奖励，同时施加 KL 散度惩罚以防止模型偏移旧模型太多。优化目标如下：

$$\max_{\pi} \mathcal{J}(\pi) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi(\cdot|x)} \left[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

这实际上是一个**带约束的优化问题**。虽然 PPO 通过迭代采样来近似求解，但在数学上，这个目标函数存在一个**解析解**。

第二阶段：最优策略的解析形式

上述目标函数本质上等价于最小化以下 KL 散度：

$$\min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \left[D_{\text{KL}} \left(\pi(y|x) \parallel \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) e^{\frac{1}{\beta} r(x,y)} \right) \right]$$

其中 $Z(x) = \sum_y \pi_{\text{ref}}(y|x) e^{\frac{1}{\beta} r(x,y)}$ 是配分函数 (Partition Function)。

证明如下：

变分法求解 $\max_{\pi} J(\pi)$

2020年1月21日 星期三 16:38

优化目标: $\max_{\pi} J(\pi) = E[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}]$
 s.t. $\sum_y \pi(y|x) = 1$

拉格朗日乘子法:

$$L(\pi, \lambda) = J(\pi) - \lambda (\sum_y \pi(y|x) - 1)$$

$$= (E[r(x, y)] - \beta \sum_y \pi(y|x) [\log \pi(y|x) - \log \pi_{\text{ref}}(y|x)]) - \lambda (\sum_y \pi(y|x) - 1)$$

$$= \sum_y [\pi(y|x) r(x, y) - \beta \pi(y|x) \log \pi(y|x) + \beta \pi(y|x) \log \pi_{\text{ref}}(y|x) - \lambda \pi(y|x)] + \lambda$$

$$\Leftrightarrow \frac{\partial L}{\partial \pi(y|x)} = 0$$

$$\Rightarrow r(x, y) - \beta (\log \pi(y|x) + 1) + \beta \log \pi_{\text{ref}}(y|x) - \lambda = 0.$$

接下来求 $\pi(y|x)$, 对上式处理.

$$\beta (\log \pi(y|x) + 1) = r(x, y) + \beta \log \pi_{\text{ref}}(y|x) - \lambda$$

$$\Rightarrow \pi(y|x) = e^{\frac{r(x, y)}{\beta}} \cdot \pi_{\text{ref}}(y|x) \cdot e^{(-1 + \frac{\lambda}{\beta})}$$

此时带回约束条件 $\sum_y \pi(y|x) = 1$

$$\sum_y e^{\frac{r(x, y)}{\beta}} \cdot \pi_{\text{ref}}(y|x) \cdot e^{(-1 + \frac{\lambda}{\beta})} = 1.$$

5分28秒

$$\Rightarrow e^{(-1 + \frac{\lambda}{\beta})} = \frac{1}{\sum_y e^{\frac{r(x, y)}{\beta}} \cdot \pi_{\text{ref}}(y|x)}$$

带回去, 可求得最优 $\pi(y|x)$

$$\Rightarrow \pi^*(y|x) = \frac{e^{\frac{r(x, y)}{\beta}} \pi_{\text{ref}}(y|x)}{\sum_y e^{\frac{r(x, y)}{\beta}} \pi_{\text{ref}}(y|x)}$$

一般令配分函数 $Z(x) = \sum_y e^{\frac{r(x, y)}{\beta}} \pi_{\text{ref}}(y|x)$.

则 $\pi^*(y|x) = \frac{1}{Z(x)} \cdot e^{\frac{r(x, y)}{\beta}} \pi_{\text{ref}}(y|x)$.

证明: 目标函数 $\max_{\pi} J(\pi) = E[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}]$

等价 $\min_{\pi} E[D_{\text{KL}}(\pi(y|x) \parallel \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) e^{\frac{r(x, y)}{\beta}})]$

其实也就是把 π 用 Z 替换.

从 $\pi^*(y|x) = \frac{1}{Z(x)} e^{\frac{r(x, y)}{\beta}} \pi_{\text{ref}}(y|x)$ 出发.

$$\Rightarrow r(x, y) = \beta (\log \pi^*(y|x) + \log Z(x) - \log \pi_{\text{ref}}(y|x))$$

代入 $J(\pi)$ 有: (可消掉最后一项)

$$\max_{\pi} E[\beta \log Z(x) - \beta \log \frac{\pi(y|x)}{\pi^*(y|x)}]$$

常数项.

因此等价于 $\min_{\pi} E[\log \frac{\pi(y|x)}{\pi^*(y|x)}]$

$$= \min_{\pi} E[D_{\text{KL}}(\pi(y|x) \parallel \pi^*(y|x))]$$

物理上的理解: 即最小化达到最佳分布策略 π^* 的期望.

图 12.6: DPO 数学原理证明

根据**吉布斯不等式**, 当且仅当两个分布相等时, KL 散度最小(为0)。因此, 对于固定的奖励函数 $r(x, y)$, **最优策略 $\pi^*(y|x)$** 必须满足: $\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) e^{\frac{1}{\beta} r(x, y)}$

Extension 12.7 吉布斯不等式

核心原理：在所有概率分布中，真实分布能让“交叉熵”或“相对熵”达到最小。

设 $P = (p_1, p_2, \dots, p_n)$ 为真实分布， $Q = (q_1, q_2, \dots, q_n)$ 为拟合的分布，都是定义在同一集合上的概率分布（ $p_i \geq 0, q_i \geq 0$ ，且和为 1）。

- $P = (p_1, p_2, \dots, p_n)$ 为真实分布
- $Q = (q_1, q_2, \dots, q_n)$ 为拟合的分布

都是定义在同一集合上的概率分布（ $p_i \geq 0, q_i \geq 0$ ，且和为 1）。

那么有： $\sum_{i=1}^n p_i \ln p_i \geq \sum_{i=1}^n p_i \ln q_i$ 并且等号成立当且仅当 $p_i = q_i$ 对所有 i 都成立。

把上面的式子移项，就得我们熟悉的东西： $\sum_{i=1}^n p_i \ln \frac{p_i}{q_i} \geq 0$

这就是我们一直接触的 **KL 散度（相对熵）**： $D_{\text{KL}}(P||Q) \geq 0$

所以实际上**吉布斯不等式 = KL 散度非负性**

用信息论的知识去直观理解：用 Q 去“假装”数据来自真实的 P ，付出的平均编码代价一定比用 P 自己编码更大或扯平。

通常在 RLHF 中，我们用神经网络去逼近这个 π^* 。但在 DPO 中，我们利用这个等式进行**代数变换**。

第三阶段：奖励函数的逆向表示

现在，我们把上面的等式反过来，用最优策略 π^* 和参考策略 π_{ref} 来表示奖励函数 $r(x, y)$ 。

对等式两边取对数：

$$\log \pi^*(y|x) = \log \left(\frac{1}{Z(x)} \pi_{\text{ref}}(y|x) e^{\frac{1}{\beta} r(x, y)} \right)$$

$$\log \pi^*(y|x) = -\log Z(x) + \log \pi_{\text{ref}}(y|x) + \frac{1}{\beta} r(x, y)$$

移项解出 $r(x, y)$ ：

$$\frac{1}{\beta} r(x, y) = \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \log Z(x)$$

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

这个公式告诉我们：**任何一个奖励函数，都可以等价地映射为一个最优策略与参考策略的对数比率，加上一个仅与 x 有关的常数项。**

第四阶段：代入 Bradley-Terry 模型

在人类偏好建模中，我们通常假设偏好服从 **Bradley-Terry (BT) 模型**。即，回答 y^+ 优于 y^- 的概率取决于两者奖励差值的 Sigmoid： $P(y^+ \succ y^- | x) = \sigma(r(x, y^+) - r(x, y^-))$

现在，我们将第三阶段推导出的 $r(x, y)$ 代入上式。注意，当我们计算 $r(x, y^+) - r(x, y^-)$ 时，**配分函数项 $\beta \log Z(x)$ 会因为相减而自动抵消（因为它只与 x 有关，与 y 无关）**：

$$\begin{aligned} r(x, y^+) - r(x, y^-) &= \left(\beta \log \frac{\pi^*(y^+|x)}{\pi_{\text{ref}}(y^+|x)} + \beta \log Z(x) \right) - \left(\beta \log \frac{\pi^*(y^-|x)}{\pi_{\text{ref}}(y^-|x)} + \beta \log Z(x) \right) \\ &= \beta \log \frac{\pi^*(y^+|x)}{\pi_{\text{ref}}(y^+|x)} - \beta \log \frac{\pi^*(y^-|x)}{\pi_{\text{ref}}(y^-|x)} \end{aligned}$$

第五阶段：最终的 DPO 损失函数

现在，我们将参数化的策略 π_θ 作为 π^* 的近似。我们的目标是最大化观测到的偏好数据的似然度。

实际中目标函数就转换为最小化负对数似然：

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

这就是 DPO 的最终公式。这个损失对每条偏好数据 (x, y^+, y^-) 都在推高输出 y^+ 相对概率、压低输出 y^- 相对概率。同时，参考模型的对数概率差作为一个常数项嵌入其中，约束 π_θ 不要偏离 π_{ref} 太远。

概括而言，DPO 的核心原理就是将“**最大化奖励减 KL**”问题等价转化为**对比两段输出谁更好的分类问题**。模型通过直接优化偏好数据的似然来隐式达到与 RLHF 相同的效果。

值得一提的是， β 超参数的选择十分重要：它决定了 DPO 更新的“温和”程度。较大的 β （如 0.5 以上）意味着模型几乎保持原状，只做细微调整；较小的 β （如 0.1）则赋予模型更大自由去迎合偏好，但过小可能造成模型过度偏离原本分布。

12.11.4. DPO 的训练

一般实现流程如下，训练的技巧可以参考一般的监督学习。

1. **准备初始模型**：选择一个经过监督微调的基础模型，并将其副本冻结作为参考模型 π_{ref} （参考策略）。待训练模型 π_θ 初始化为相同权重。
2. **组建数据集**：加载偏好数据集，每条包含（输入，更偏爱的回复，更不喜欢的回复）（也就是 (x, y^+, y^-) ）。在数据加载时，先将文本转换为模型可处理的 token 序列格式，通常需要将两段输出拼接在相同的 prompt 下分别形成完整的输入-输出序列用于计算概率。比如 Anthropic 公司开源的数据集里的这种：

```
"chosen": "\n\nHuman: Do you know why xxxxx?\n\nAssistant: To xxxxxxxxxxxxxxxx.",
"rejected":
↪ "\n\nHuman: Do you know why xxxxx?\n\nAssistant: I know xxxxxxxxxxxxxxxxxxxx?"
```

3. **定义 DPO 损失**：按照我们上文推导的 DPO 损失函数，其实主要就改一下参数 β 。
4. **参考模型处理**：参考模型在实现中一般不需要显式保存整个模型的副本。更高效的做法是在计算损失时，将当前模型输出与初始模型输出对比即可（因为参考模型权重等于初始模型）。
5. **迭代训练**：对于整个偏好数据集进行多轮遍历（epoch），关注验证集性能变化。

12.11.5. DPO、PPO、GRPO 三者比较

维度	PPO	DPO	GRPO
算法类型	强化学习 (RL)	监督学习 (非传统 RL)	强化学习 (PPO 变体)
是否需要奖励模型	需要	不需要	不需要显式
训练信号来源	奖励模型打分	人类/模型偏好对 (chosen vs rejected)	同一 prompt 下多回答的相对好坏
优化目标	最大化期望奖励 + KL 约束	直接最大化偏好概率	最大化组内相对优势
是否在线采样	是	否 (离线)	是
训练稳定性	中等 (需调参)	很稳定	比 PPO 稳定
实现复杂度	高	低	中
计算/显存开销	高	低	中
典型应用	ChatGPT 早期 RLHF	SFT 后对齐主流方案	大模型 RL 对齐 (DeepSeek 系)

12.11.6. 其他相关前沿研究

- **多目标偏好优化 (Multi-objective DPO)**：实际应用中，往往需要同时对齐模型的**多个方面**，比如既要助人为乐，又要风格幽默，还要事实准确等等。标准 DPO 只能处理**综合后的单一偏好判断**。为此，研究者提出一些方案：
 - **MODPO (Multi-Objective Direct Preference Optimization)**：它的核心思想是：如果我们有多个偏好维度的反馈数据（例如安全性、礼貌性分别打分），可以将这些目标通过加权组合成一个总的偏好打分，然后将 DPO 中的奖励函数 $r(x, y)$ 替换为这个加权总分。训练时动态调整各目标权重，使得模型在不同目标上都达到要求，而非只偏好某一个方面。MODPO 等于是把**多目标权衡融入损失函数**，以较小改动实现“一次微调，多目标对齐”。
 - **分阶段或多任务偏好优化**：另一种思路是**拆分训练过程**。例如先用 DPO 优化模型的有用性，后用另一套偏好数据优化它的安全性。或者设计多任务损失，部分是 DPO 损失（偏好比较），部分是常规的 SFT 损失（对某些约束任务）。一些工程实践中，会先对模型在安全守则上做 RLHF 微调，再对其它方面用 DPO。
 - **值得注意的是**：多目标偏好有时会出现**偏好冲突**（比如“准确回答” vs “隐私保护”，有时详细准确可能侵犯隐私）。在这种情况下，简单的加权可能不够，需要制定**优先级**或引入更高级的决策机制。
- **多轮对话偏好优化**：DPO 主要聚焦于**单轮回复**的质量提升。但对于连续多轮对话，如何让模型遵循长程偏好（比如始终礼貌、一致的人格等）是更复杂的问题。有几种尝试：
 - **对整段对话进行偏好比较**：收集整个多轮对话 sessions 的偏好评价，而不是单轮。比如让人工比较两段完整对话记录哪个更让人满意。然后将整个对话当作一个“大输出”应用 DPO。但这样做需要模型能生成完整

对话，实现上难度较高，而且优化长文本的策略也未必等价于逐轮优化。

- **逐轮优化 + 记忆**：另一方法是仍然逐轮应用 DPO，但在模型架构或提示中加入记忆或隐藏状态，使模型知道之前的表现，从而在损失中反映“延续性”偏好。比如在每轮输入里包含上文摘要，并设计偏好反馈涵盖“有没有自相矛盾”之类的指标。

12.12. 专家迭代实验 (Expert Iteration)

12.12.1. 传统 SFT (Supervised Fine-Tuning)

- 直接使用标准答案 (ground truth) 进行**监督学习**。
- 模型只学习 “正确答案”，不参与探索过程。
- **优点**：简单直接。
- **缺点**：无法利用模型自身探索能力，数据质量完全依赖人工标注。

12.12.2. 专家迭代 (Expert Iteration)

专家迭代的核心思想是：

模型自己探索 → 筛选优质答案 → 用优质答案正反馈训练 → 模型变强 → 继续探索

具体流程：

1. 先自己尝试解题 (**生成多个答案**)
2. 找出做对的题 (**过滤**)
3. 只练习做对的题 (SFT 训练)
4. 下一轮更可能做对更多题。

12.12.3. 详细工程步骤

1. Step 0: 初始化和基准评估

- 初始化 vLLM 框架
- 在一切训练开始前：
 - 先跑完整验证集
 - 记录 baseline 性能
 - 作为后续对比基准

2. Step 1: 数据抽样

- 从 ‘train.jsonl’ 抽样问题
- 控制批量规模 (避免 rollout 成本过高)

3. Step 2: Rollout 生成

- 使用当前专家模型
- 对每个问题生成多个回答
- 形成候选答案集合

4. Step 3: 奖励函数过滤 使用 ground truth 和评分函数进行打分和筛选。

5. Step 4: SFT 训练

- 使用 transformers 加载模型到训练卡
- 使用筛选后的高质量数据集
- 执行 SFT 微调

- 产出新权重

6. Step 5: 更新 vLLM 权重 (关键优化)

- 避免每轮重复初始化 vLLM
- 每轮训练后：
 - 直接更新权重
 - 不重新构建推理引擎

7. Step 6: 验证集评估

- 每轮迭代后：跑验证集, 记录性能变化
- 最终：完整评估验证集, 输出最终结果

实验日志：

1. 模型性能不佳，导致每一轮输出的多个答案都被过滤掉了，即输出质量差，奖励低。还是要至少确保每一组 rollout 都起码有一个被保留。以前是“只收 $\text{strict reward} > 0$ ”，太容易全空。现在改为“严格 + 宽松双层”，提取后结果再判断。
2. 模型与推理引擎初始化。训练模型用 transformers 加载到训练卡。生成模型用 vLLM 初始化到评估卡，并做了 xformers 相关 patch，目的是让 V100 上更稳定运行。

12.13. 第七章大作业 GRPO 训练循环

12.13.1. GRPO 训练中的易混淆的参数

- **采样层 (rollout)**: 先拿问题去生成回答数据
- **训练层 (epoch/batch)**: 对这批回答做几轮训练
- **计算层 (microbatch)**: 为了省显存做梯度累积

变量	含义	所在层级
n_grpo_steps	外层训练总步数 (重复 “采样 + 训练” 多少次)	最外层
group_size	每个 prompt 采样多少个回答	采样层
rollout_batch_size	每个 GRPO step 一共采样多少条回答 (responses)	采样层
n_prompts_per_rollout_batch	每步采样多少个 prompt, 公式: $\text{rollout_batch_size} // \text{group_size}$	采样层
epochs_per_rollout_batch	对同一批 rollout 数据重复训练几遍	训练层
train_batch_size	每次优化器更新前, 处理的 “大 batch” 大小	训练层
gradient_accumulation_steps	一个大 batch 内分成多少个 microbatch 来累积梯度	计算层
micro_train_batch_size	每个 microbatch 大小, 公式: $\text{train_batch_size} // \text{gradient_accumulation_steps}$	计算层

举个例子, 假设:

- $\text{n_grpo_steps} = 3$
- $\text{rollout_batch_size} = 24$
- $\text{group_size} = 4$
- $\text{epochs_per_rollout_batch} = 2$
- $\text{train_batch_size} = 12$
- $\text{gradient_accumulation_steps} = 3$

那么:

1. 每个 GRPO step 先**采样**

- $\text{n_prompts_per_rollout_batch} = 24 / 4 = 6$
- 即: 抽 6 个题目, 每题生成 4 个回答, 共 24 条 rollout

2. 然后**训练** (同一批 24 条数据)

- $\text{epochs_per_rollout_batch} = 2$, 所以这 24 条会训练 2 遍
- 每遍按 $\text{train_batch_size} = 12$ 切成 2 个大 batch

3. 每个大 batch 再做**梯度累积**

- $\text{micro_train_batch_size} = 12/3 = 4$
- 每个大 batch 分成 3 个 microbatch (每个 4 条)

4. 所以每个 GRPO step 的**优化器更新数**

- 每个 epoch: $24/12 = 2$ 次更新
- 每个 step: $2(\text{epoch}) * 2 = 4$ 次更新

5. 全训练总计

- 总 rollout 数: $3 * 24 = 72$
- 总 optimizer 更新: $3 * 4 = 12$

12.13.2. 完整训练思路

完整 GRPO train loop 按下面链路实现：

1. 每个训练 step 先从 train.jsonl 采样 $n_prompts_per_rollout_batch = rollout_batch_size / group_size$ 个题目。
2. 用 vLLM 对每题生成 $group_size$ 个 rollout (stop 到 `</answer>`)。
3. 计算每条 rollout 的 raw reward, 再按组做**中心化/标准差归一化**得到 advantage。
4. 把 prompt + response 转成训练张量 (input_ids/labels/response_mask)。
5. 用 policy 算 response token 的 log-prob。
6. 按 gradient_accumulation_steps 做微批训练，支持：
7. 做梯度裁剪和 optimizer step。
8. 每 val_every_steps 做一次验证奖励评估，记录曲线。

12.13.3. 实验中真实遇到的问题

1. 我是在 V100 上跑的，要更改代码使得其兼容 vllm 架构；
2. Qwen-1.5B-Math 模型还是能力不够，如果要正经按照要求严格限制输出规范和答案的话，**奖励非常稀疏**，约等于 0，即每个批次一个正确答案都得不到，模型失去了训练的意义。因此，我放宽了约束，侧重答案正确，哪怕格式不规范也不要紧。
3. 我实验最终输出的结果依旧不咋地，即使放宽了约束，也只是从一个问题都回答不上来变成了回答上来一两个。。